

Laporan Tugas Besar

Teori Bahasa Formal dan Automata

Milestone 3 - Semantic Analysis



Disusun oleh kelompok :
Kerjakan J - 5

Mahatma Brahmana	13524015
Benedict Darrel Setiawan	13524057
Raymond Jonathan Dwi Putra J	13524059
Reynard Anderson Wijaya	13524111

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

Daftar Isi

Daftar Isi.....	2
BAB 1 Landasan Teori.....	4
1.1 Semantic Analysis.....	4
1.2 Abstract Syntax Tree (AST).....	4
1.3 Syntax-Directed Translation dan Attributed Grammar.....	6
1.4 Decorated Abstract Syntax Tree.....	6
1.5 Symbol Table.....	7
1.6 Scope Resolution.....	8
1.7 Type Checking dan Type Compatibility.....	9
1.7.1 Type Checking.....	9
1.7.2 Type Compatibility.....	9
1.8 Semantic Error.....	10
BAB 2 Perancangan Program.....	12
2.1 Gambaran Umum dan Teknologi yang Digunakan.....	12
2.2 Struktur Program dan Fungsi/Kelas Utama.....	13
2.2.1 Fungsi dan Modul Utama Program pada Semantic Analysis.....	14
2.3 Alur Kerja.....	16
2.4 Rancangan AST dan Decorated AST.....	18
2.5 Rancangan Symbol Table.....	19
2.6 Rancangan Semantic Checking.....	21
BAB 3 Implementasi.....	24
3.1 Implementasi Program Pembentukan AST.....	24
3.2 Implementasi Node Pencarian Child.....	24
3.3 Implementasi Node Variabel.....	24
3.4 Implementasi Node Factor.....	25
3.5 Implementasi Node Term.....	25
3.6 Implementasi Node Simple Expression.....	26
3.7 Implementasi Node Expression.....	26
3.8 Implementasi Node Type.....	26
3.9 Implementasi Node Constant.....	27
3.10 Implementasi Parameter Formal.....	27
3.11 Implementasi Statement.....	28
3.11.1 Implementasi Node Assignment Statement.....	28
3.11.2 Implementasi Node Procedure/Function Call.....	29
3.11.3 Implementasi Node If Statement.....	29
3.11.4 Implementasi Node While Statement.....	29
3.11.5 Implementasi Node Repeat Statement.....	30
3.11.6 Implementasi Node For Statement.....	30
3.11.7 Implementasi Node Case Statement.....	30

3.12 Implementasi Statement List.....	31
3.13 Implementasi Declaration Part.....	31
3.13.1 Implementasi Node Const Declaration.....	32
3.13.2 Implementasi Node Type Declaration.....	32
3.13.3 Implementasi Node Variable Declaration.....	32
3.13.4 Implementasi Node Procedure Declaration.....	32
3.13.5 Implementasi Node Function Declaration.....	33
3.14 Implementasi Symbol Table.....	33
3.14.1 Implementasi Deklarasi Symbol.....	34
3.14.2 Implementasi Scope.....	34
3.14.3 Implementasi Lookup Symbol.....	34
3.14.4 Implementasi TAB, ATAB, dan BTAB.....	35
3.15 Implementasi Semantic Analyzer.....	35
3.15.1 Implementasi Visit Function.....	36
3.15.2 Implementasi Analisis Program.....	36
3.15.3 Implementasi Analisis Deklarasi.....	37
3.15.4 Implementasi Analisis Statement.....	37
3.15.5 Implementasi Inferensi Tipe Ekspresi.....	38
3.16 Implementasi Pengecekan Semantik.....	38
3.16.1 Implementasi Pengecekan Deklarasi Ganda.....	39
3.16.2 Implementasi Pengecekan Identifier.....	39
3.16.3 Implementasi Pengecekan Assignment.....	39
3.16.4 Implementasi Pengecekan Operator.....	40
3.16.5 Implementasi Pengecekan Struktur Kontrol.....	40
3.16.6 Implementasi Scope Procedure dan Function.....	40
3.16.7 Implementasi Pengecekan Pemanggilan Procedure dan Function.....	41
3.16.8 Implementasi Pengecekan Array.....	42
3.16.9 Implementasi Pengecekan Record.....	42
3.16.10 Implementasi Pengecekan Subrange dan Case.....	43
3.17 Implementasi Output Semantic Analysis.....	43
3.17.1 Implementasi Output Decorated AST.....	44
3.17.2 Implementasi Output Symbol Table.....	44
3.17.3 Implementasi Output Error.....	44
BAB 4 Pengujian.....	45
4.1 Pengujian Masukkan dan Keluaran Program.....	45
BAB 5	
Kesimpulan dan Saran.....	53
5.1 Kesimpulan.....	53
5.2 Saran.....	53
Lampiran.....	54
Daftar Pustaka.....	55

BAB 1 Landasan Teori

1.1 Semantic Analysis

Semantic analysis merupakan tahap lanjutan setelah proses lexical analysis dan syntax analysis dalam proses kompilasi program. Pada tahap lexical analysis, source code diubah menjadi kumpulan token, sedangkan pada tahap syntax analysis, token-token tersebut diperiksa berdasarkan aturan grammar untuk membentuk struktur program yang valid secara sintaksis. Namun, valid secara sintaksis belum tentu berarti program tersebut valid secara makna. Oleh karena itu, diperlukan tahap semantic analysis untuk memastikan bahwa program yang telah dibuat memiliki makna yang benar sesuai aturan bahasa pemrograman.

Semantic analysis berfungsi untuk memeriksa kesesuaian makna dari suatu program. Tahap ini memastikan bahwa setiap komponen program tidak hanya tersusun sesuai grammar, tetapi juga digunakan dengan benar. Sebagai contoh, suatu ekspresi mungkin valid secara sintaksis, tetapi dapat salah secara semantik apabila mencoba menjumlahkan nilai bertipe **String** dengan nilai bertipe **Integer**, atau membandingkan nilai bertipe **Char** dengan **String**.

Dalam proses semantic analysis, terdapat beberapa bentuk pengecekan utama yang dilakukan. Pertama, **type checking** atau pengecekan tipe, yaitu proses untuk memastikan bahwa operator dan operand yang digunakan dalam suatu ekspresi memiliki tipe data yang kompatibel. Misalnya, operasi aritmatika seharusnya dilakukan pada tipe numerik, sedangkan operasi logika seharusnya dilakukan pada tipe Boolean. Kedua, **symbol table management** atau pengelolaan tabel simbol, yaitu proses untuk memastikan bahwa setiap identifier, seperti variabel, konstanta, fungsi, atau prosedur, telah dideklarasikan sebelum digunakan dan tidak dideklarasikan ulang pada cakupan yang sama.

Selain itu, semantic analysis juga melakukan **scope resolution** atau pengecekan lingkup. Scope resolution digunakan untuk menentukan apakah suatu identifier dapat diakses pada bagian program tertentu berdasarkan hierarki blok kode. Sebagai contoh, variabel yang dideklarasikan di dalam suatu prosedur hanya dapat digunakan di dalam prosedur tersebut, sedangkan variabel global dapat digunakan pada cakupan yang lebih luas. Tahap lain yang dapat dilakukan adalah **control flow validation**, yaitu pengecekan terhadap alur kontrol program agar tetap logis. Contohnya adalah memastikan bahwa instruksi seperti **break** atau **continue** hanya digunakan di dalam struktur perulangan, serta memastikan fungsi memiliki nilai kembalian yang sesuai apabila dideklarasikan memiliki return value.

Hasil dari semantic analysis dapat berupa struktur pohon yang telah diberi anotasi, yaitu **decorated abstract syntax tree**. Anotasi tersebut dapat berupa informasi tambahan seperti tipe data suatu ekspresi, referensi identifier terhadap symbol table, informasi scope, serta atribut lain yang diperlukan untuk tahap kompilasi berikutnya. Dengan demikian, semantic analysis berperan penting dalam memastikan bahwa program tidak hanya benar secara sintaksis, tetapi juga benar secara semantik.

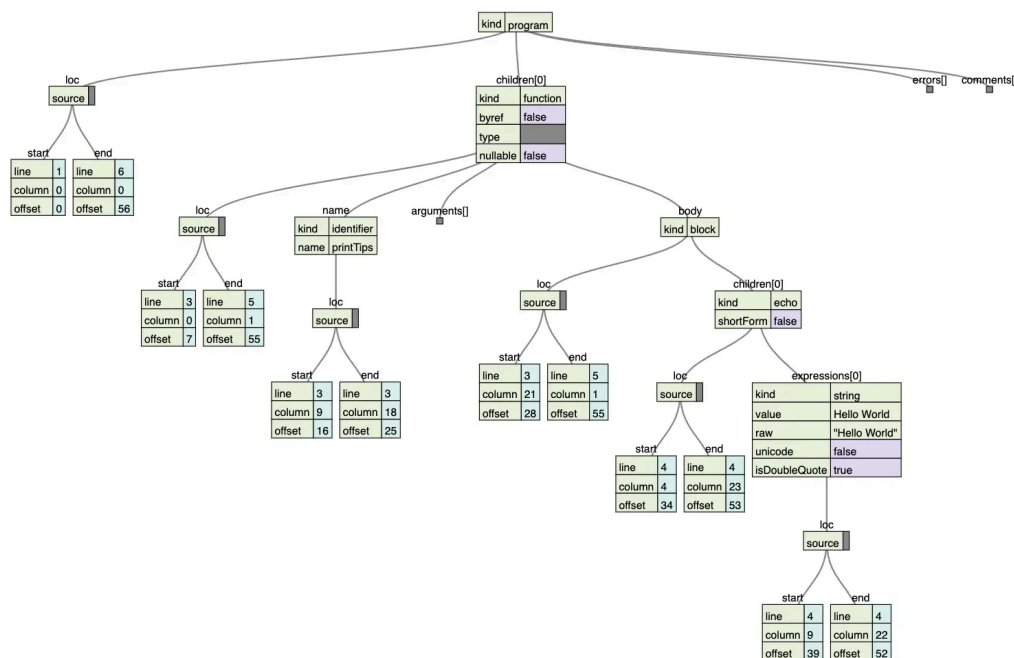
1.2 Abstract Syntax Tree (AST)

Abstract Syntax Tree atau **AST** merupakan representasi struktur program dalam bentuk pohon yang lebih ringkas dibandingkan parse tree. Parse tree biasanya merepresentasikan seluruh aturan grammar yang digunakan oleh parser, termasuk simbol terminal, non-terminal, tanda baca, dan elemen

sintaksis lain yang dibutuhkan untuk proses pengecekan syntax. Namun, tidak semua informasi tersebut diperlukan pada tahap analisis lanjutan seperti semantic analysis. Oleh karena itu, AST dibentuk dengan hanya menyimpan informasi yang penting untuk memahami struktur dan makna program.

Pada parse tree, struktur program sangat bergantung pada aturan grammar yang digunakan. Setiap node pada parse tree merepresentasikan bagian dari proses derivasi grammar, sehingga bentuk pohonnya cenderung lebih detail dan panjang. Sebaliknya, AST lebih berfokus pada hubungan semantik antarbagian program. Misalnya, dalam suatu ekspresi aritmatika, AST akan lebih menekankan hubungan antara operator dan operand, bukan lagi detail sintaksis seperti tanda kurung atau aturan produksi grammar yang digunakan untuk membentuk ekspresi tersebut. Sebagai contoh, ekspresi `a + b * c` pada parse tree dapat memiliki banyak node yang merepresentasikan aturan grammar seperti `expression`, `term`, dan `factor`. Pada AST, ekspresi tersebut dapat direpresentasikan secara lebih sederhana dengan operator `+` sebagai node utama, operand kiri berupa `a`, dan operand kanan berupa operasi `*` antara `b` dan `c`. Dengan bentuk seperti ini, AST menjadi lebih mudah digunakan untuk melakukan pengecekan semantik, seperti menentukan tipe data hasil ekspresi atau memastikan kompatibilitas antara operator dan operand.

Dalam konteks semantic analysis, AST menjadi struktur penting karena digunakan sebagai dasar untuk melakukan proses anotasi dan pengecekan makna. Setiap node pada AST dapat diberi informasi tambahan seperti tipe data, referensi ke symbol table, dan informasi lingkup identifier. Setelah proses anotasi dilakukan, AST tersebut dapat berkembang menjadi decorated AST. Dengan demikian, AST berfungsi sebagai representasi antara yang menghubungkan hasil syntax analysis dengan proses semantic analysis dan tahap kompilasi berikutnya.



Gambar 1.1 Contoh Abstract Syntax Tree (AST)

(sumber : <https://medium.com/@milhamsuryapratama/abstract-syntax-tree-ast-91440fbca59f>)

1.3 Syntax-Directed Translation dan Attributed Grammar

Syntax-Directed Translation merupakan metode penerjemahan struktur program dengan memanfaatkan aturan grammar yang telah didefinisikan. Pada metode ini, setiap aturan produksi grammar tidak hanya digunakan untuk mengenali struktur sintaksis program, tetapi juga dapat dilengkapi dengan aksi semantik atau *semantic action*. Aksi semantik tersebut digunakan untuk membentuk informasi tambahan yang diperlukan pada tahap kompilasi berikutnya, seperti pembentukan node AST, penentuan tipe data, dan pengelolaan informasi identifier.

Pada proses semantic analysis, Syntax-Directed Translation digunakan untuk mengubah parse tree menjadi Abstract Syntax Tree yang lebih ringkas. Parse tree yang dihasilkan dari tahap syntax analysis masih mengandung banyak informasi sintaksis yang tidak selalu diperlukan, seperti simbol terminal tertentu, tanda kurung, titik koma, atau node grammar yang hanya berfungsi sebagai penghubung. Melalui semantic action, informasi-informasi yang tidak diperlukan tersebut dapat dihilangkan, sedangkan informasi penting seperti operator, operand, deklarasi, statement, dan ekspresi tetap dipertahankan.

Attributed Grammar merupakan grammar yang dilengkapi dengan atribut pada setiap simbol grammar dan aturan semantik untuk menghitung nilai atribut tersebut. Atribut ini dapat digunakan untuk menyimpan informasi yang berkaitan dengan makna suatu node, seperti tipe data, nama identifier, nilai konstanta, atau referensi ke symbol table. Dengan adanya atribut, grammar tidak hanya berperan dalam membentuk struktur sintaksis, tetapi juga membantu proses pengecekan makna dari program.

Pada implementasi semantic analysis, attributed grammar digunakan untuk menelusuri struktur parse tree atau AST dan menghitung atribut dari setiap node. Misalnya, pada ekspresi $a + b$, semantic analyzer dapat memeriksa tipe data dari a dan b , lalu menentukan tipe hasil dari operasi $+$. Jika kedua operand memiliki tipe yang kompatibel, node ekspresi dapat diberi atribut tipe hasil. Sebaliknya, jika tipe operand tidak kompatibel, semantic analyzer akan menghasilkan semantic error.

Proses semantic analysis dapat dilakukan dengan pendekatan **L-Attributed Grammar**. Artinya, informasi atribut dapat dihitung dengan memanfaatkan atribut dari parent atau sibling yang berada di sebelah kiri. Pendekatan ini cocok digunakan dalam penelusuran top-down karena proses analisis dapat dilakukan secara bertahap dari node induk menuju node anak. Pada proses ini, fungsi visitor dapat digunakan untuk mengunjungi setiap node, menjalankan semantic action yang sesuai, memperbarui symbol table, serta memberikan anotasi pada node yang sedang diproses.

1.4 Decorated Abstract Syntax Tree

Decorated Abstract Syntax Tree merupakan Abstract Syntax Tree yang telah diberi anotasi tambahan hasil dari proses semantic analysis. Jika AST hanya merepresentasikan struktur utama program secara ringkas, maka Decorated AST menyimpan informasi semantik yang lebih lengkap pada setiap node-nya. Informasi tambahan tersebut dapat berupa tipe data, referensi identifier terhadap symbol table, informasi scope, nilai konstanta, atau atribut lain yang diperlukan untuk tahap kompilasi berikutnya.

Proses pembentukan Decorated AST dilakukan setelah AST berhasil dibentuk dari parse tree. Semantic analyzer akan menelusuri setiap node pada AST, kemudian melakukan pemeriksaan sesuai jenis

node tersebut. Misalnya, pada node assignment, semantic analyzer akan memeriksa apakah variabel tujuan sudah dideklarasikan dan apakah tipe data ekspresi di sebelah kanan sesuai dengan tipe variabel di sebelah kiri. Jika pemeriksaan berhasil, node assignment dapat diberi anotasi berupa tipe data dan referensi ke entri symbol table yang sesuai.

Decorated AST juga berperan dalam proses pengecekan identifier. Ketika semantic analyzer menemukan penggunaan suatu identifier, sistem akan melakukan pencarian pada symbol table untuk memastikan bahwa identifier tersebut telah dideklarasikan sebelumnya. Jika ditemukan, node identifier pada AST dapat diberi anotasi berupa referensi ke entri symbol table. Referensi ini berguna agar informasi seperti tipe data, jenis objek, dan level scope dari identifier tersebut dapat digunakan kembali pada proses analisis berikutnya. Selain itu, Decorated AST digunakan untuk menyimpan hasil pengecekan tipe pada ekspresi. Pada ekspresi aritmatika, logika, atau relasional, setiap node ekspresi dapat diberi anotasi berupa tipe hasil operasi. Sebagai contoh, operasi antara dua nilai bertipe **Integer** dapat menghasilkan tipe **Integer**, sedangkan operasi relasional seperti $a < b$ akan menghasilkan tipe **Boolean**. Dengan adanya anotasi ini, semantic analyzer dapat memastikan bahwa ekspresi digunakan pada konteks yang sesuai, misalnya kondisi pada **if** atau **while** harus bernilai **Boolean**.

1.5 Symbol Table

Symbol table merupakan struktur data yang digunakan dalam compiler untuk menyimpan informasi mengenai simbol atau identifier yang muncul di dalam program. Identifier tersebut dapat berupa nama variabel, konstanta, fungsi, prosedur, tipe data, parameter, maupun elemen lain yang dideklarasikan oleh programmer. Informasi yang tersimpan dalam symbol table digunakan oleh compiler untuk melakukan pemeriksaan semantik dan membantu proses kompilasi pada tahap berikutnya. Pada proses kompilasi, symbol table mulai digunakan ketika compiler menemukan deklarasi identifier di dalam source code. Setiap identifier yang dideklarasikan akan dicatat ke dalam symbol table bersama dengan atribut-atribut yang berkaitan dengannya. Atribut tersebut dapat berupa nama identifier, jenis identifier, tipe data, lokasi deklarasi, lingkup atau scope, jumlah parameter pada fungsi, tipe nilai kembalian, serta informasi tambahan lain yang diperlukan oleh compiler.

Symbol table memiliki peran penting dalam memastikan bahwa setiap identifier digunakan dengan benar. Ketika suatu identifier digunakan dalam ekspresi atau statement, compiler akan melakukan pencarian terhadap identifier tersebut di dalam symbol table. Jika identifier ditemukan, compiler dapat mengambil informasi yang diperlukan, seperti tipe data atau jenis identifier. Namun, jika identifier tidak ditemukan, maka program dapat dianggap memiliki kesalahan semantik karena identifier tersebut digunakan sebelum dideklarasikan. Selain itu, symbol table juga digunakan untuk mendeteksi deklarasi ganda. Dalam banyak bahasa pemrograman, sebuah identifier tidak boleh dideklarasikan lebih dari satu kali pada lingkup yang sama. Dengan adanya symbol table, compiler dapat memeriksa apakah suatu nama identifier sudah pernah digunakan sebelumnya dalam scope yang sama. Jika sudah ada, maka compiler dapat memberikan pesan error bahwa terjadi redeclaration atau deklarasi ulang.

Symbol table juga membantu dalam proses type checking. Misalnya, ketika terdapat operasi assignment seperti $x := y + 1$, compiler perlu mengetahui tipe data dari x dan y . Informasi tersebut diperoleh dari symbol table. Jika tipe data antara variabel tujuan dan ekspresi tidak kompatibel, maka compiler

dapat mendeteksi adanya kesalahan tipe. Dengan demikian, symbol table menjadi dasar penting dalam pengecekan kebenaran makna suatu program. Secara umum, symbol table dapat diimplementasikan menggunakan berbagai struktur data, seperti array, linked list, hash table, stack, atau kombinasi beberapa struktur data. Pemilihan struktur data bergantung pada kebutuhan compiler, terutama dalam hal kecepatan pencarian, kemudahan penambahan data, dan pengelolaan scope. Pada compiler modern, symbol table biasanya dirancang agar dapat mendukung pencarian identifier secara efisien, terutama ketika program memiliki banyak deklarasi dan beberapa tingkat scope.

1.6 Scope Resolution

Scope resolution merupakan proses dalam compiler untuk menentukan lingkup keberlakuan suatu identifier di dalam program. Scope atau lingkup menyatakan bagian program tempat suatu identifier dapat dikenali dan digunakan. Dengan adanya scope, bahasa pemrograman dapat mengatur agar suatu variabel, fungsi, atau identifier lain hanya dapat diakses pada bagian program tertentu sesuai tempat deklarasinya. Dalam sebuah program, identifier dapat dideklarasikan pada berbagai tingkat lingkup. Misalnya, variabel yang dideklarasikan di luar fungsi biasanya memiliki scope global, sehingga dapat diakses oleh banyak bagian program. Sebaliknya, variabel yang dideklarasikan di dalam fungsi atau blok tertentu memiliki scope lokal, sehingga hanya dapat digunakan di dalam fungsi atau blok tersebut. Perbedaan ini penting untuk menjaga keteraturan program dan mencegah konflik nama antar identifier.

Scope resolution dilakukan ketika compiler menemukan penggunaan identifier di dalam source code. Compiler harus menentukan deklarasi mana yang dimaksud oleh penggunaan identifier tersebut. Jika suatu identifier digunakan di dalam sebuah blok lokal, compiler biasanya akan mencari deklarasinya mulai dari scope terdalam terlebih dahulu. Jika tidak ditemukan, pencarian dilanjutkan ke scope luar, hingga mencapai scope global. Jika identifier tetap tidak ditemukan, maka compiler dapat menghasilkan error karena identifier tersebut belum dideklarasikan.

Salah satu konsep penting dalam scope resolution adalah **shadowing**. Shadowing terjadi ketika identifier pada scope lokal memiliki nama yang sama dengan identifier pada scope luar. Dalam kondisi ini, identifier pada scope lokal akan menutupi identifier pada scope luar selama berada di dalam lingkup lokal tersebut. Sebagai contoh, jika terdapat variabel global bernama *x*, lalu di dalam suatu fungsi juga dideklarasikan variabel lokal bernama *x*, maka penggunaan *x* di dalam fungsi akan merujuk pada variabel lokal, bukan variabel global.

Scope resolution juga berkaitan erat dengan struktur blok dalam bahasa pemrograman. Bahasa yang mendukung blok seperti fungsi, prosedur, loop, atau statement bersarang biasanya memiliki aturan scope yang lebih kompleks. Setiap kali compiler memasuki blok baru, compiler perlu mengenali bahwa terdapat lingkup baru yang dapat memiliki deklarasi identifier sendiri. Ketika keluar dari blok tersebut, identifier yang hanya berlaku di dalam blok lokal tidak lagi dapat digunakan. Proses scope resolution sangat penting dalam menjaga konsistensi dan kejelasan program. Dengan scope resolution, compiler dapat memastikan bahwa setiap identifier merujuk pada deklarasi yang tepat. Hal ini juga mencegah kesalahan seperti penggunaan variabel di luar lingkungannya, konflik nama yang tidak valid, atau akses terhadap identifier yang belum tersedia.

1.7 Type Checking dan Type Compatibility

1.7.1 Type Checking

Type checking merupakan proses dalam compiler yang bertujuan untuk memeriksa kesesuaian tipe data pada suatu program. Proses ini memastikan bahwa setiap operasi, ekspresi, assignment, pemanggilan fungsi, dan penggunaan variabel dilakukan dengan tipe data yang sesuai. Dengan adanya type checking, compiler dapat mendeteksi kesalahan penggunaan tipe data sebelum program dijalankan. Setiap nilai di dalam program biasanya memiliki tipe data tertentu, seperti **Integer**, **Real**, **Char**, **Boolean**, atau **String**. Tipe data tersebut menentukan operasi apa saja yang dapat dilakukan terhadap nilai tersebut. Misalnya, operasi aritmatika seperti penjumlahan dan perkalian umumnya hanya dapat dilakukan pada tipe numerik, sedangkan operasi logika seperti **and**, **or**, dan **not** hanya dapat dilakukan pada tipe Boolean.

Type checking dilakukan dengan melihat tipe dari setiap komponen program. Sebagai contoh, pada ekspresi $a + b$, compiler perlu mengetahui tipe data dari a dan b . Jika keduanya bertipe numerik, maka operasi tersebut dapat dianggap valid. Namun, jika salah satu operand bertipe **String** dan operand lainnya bertipe **Integer**, maka compiler dapat menghasilkan error karena operasi tersebut tidak sesuai dengan aturan tipe data. Selain pada ekspresi, type checking juga dilakukan pada statement assignment. Pada statement seperti $x := y + 1$, compiler harus memastikan bahwa tipe data dari ekspresi $y + 1$ sesuai dengan tipe data variabel x . Jika x bertipe **Integer** dan hasil ekspresi juga bertipe **Integer**, maka assignment tersebut valid. Namun, jika x bertipe **Boolean** sedangkan ekspresi menghasilkan nilai numerik, maka terjadi kesalahan tipe. Type checking juga penting dalam pemanggilan fungsi atau prosedur. Compiler perlu memastikan bahwa jumlah argumen yang diberikan sesuai dengan jumlah parameter yang dideklarasikan. Selain itu, tipe data setiap argumen juga harus sesuai dengan tipe parameter yang diharapkan. Jika suatu fungsi membutuhkan parameter bertipe **Integer**, tetapi argumen yang diberikan bertipe **String**, maka pemanggilan fungsi tersebut tidak valid secara semantik.

Secara umum, type checking dapat dibagi menjadi dua jenis, yaitu static type checking dan dynamic type checking. Static type checking dilakukan pada saat kompilasi sebelum program dijalankan, sedangkan dynamic type checking dilakukan pada saat program berjalan. Dalam compiler, type checking umumnya dilakukan secara statis agar kesalahan tipe dapat ditemukan lebih awal.

1.7.2 Type Compatibility

Type compatibility merupakan aturan yang menentukan apakah dua tipe data dapat digunakan bersama dalam suatu operasi atau statement. Jika type checking adalah proses pemeriksaannya, maka type compatibility adalah aturan yang digunakan sebagai dasar pemeriksaan tersebut. Aturan ini menentukan apakah suatu tipe data dapat dipasangkan, dibandingkan, dioperasikan, atau diberikan nilainya kepada tipe data lain.

Dalam operasi assignment, type compatibility digunakan untuk menentukan apakah nilai di sebelah kanan dapat disimpan ke variabel di sebelah kiri. Sebagai contoh, jika suatu variabel bertipe **Integer**, maka nilai yang diberikan kepadanya seharusnya juga bertipe **Integer** atau tipe lain yang

dianggap kompatibel oleh aturan bahasa tersebut. Jika nilai yang diberikan bertipe **String**, maka assignment tersebut tidak kompatibel. Pada operasi aritmatika, type compatibility menentukan apakah operand yang digunakan dapat diproses oleh operator tertentu. Operator seperti **+**, **-**, *****, dan **/** umumnya digunakan untuk tipe numerik. Oleh karena itu, operand bertipe **Integer** dan **Real** biasanya dapat dianggap kompatibel untuk operasi aritmatika, tergantung aturan bahasa pemrogramannya. Sebaliknya, operand bertipe **Boolean** biasanya tidak kompatibel dengan operator aritmatika. Pada operasi logika, type compatibility memastikan bahwa operand yang digunakan memiliki tipe **Boolean**. Misalnya, ekspresi **a and b** hanya valid apabila **a** dan **b** sama-sama bernilai **Boolean**. Jika salah satu operand bukan **Boolean**, maka ekspresi tersebut tidak kompatibel secara tipe.

Type compatibility juga berlaku pada operasi relasional, seperti **=**, **<>**, **<**, **>**, **<=**, dan **>=**. Operator relasional biasanya digunakan untuk membandingkan dua nilai yang memiliki tipe data sejenis atau kompatibel. Hasil dari operasi relasional umumnya berupa nilai **Boolean**. Sebagai contoh, ekspresi **x > y** akan valid apabila **x** dan **y** dapat dibandingkan, misalnya sama-sama bertipe numerik. Selain itu, type compatibility juga digunakan dalam struktur kontrol program. Ekspresi kondisi pada statement seperti **if**, **while**, atau **repeat-until** biasanya harus menghasilkan nilai **Boolean**. Hal ini karena struktur kontrol membutuhkan kondisi yang hanya memiliki dua kemungkinan nilai, yaitu benar atau salah. Jika ekspresi kondisi menghasilkan nilai selain **Boolean**, maka statement tersebut tidak valid secara semantik.

1.8 Semantic Error

Semantic error merupakan kesalahan yang terjadi ketika suatu program memiliki struktur sintaksis yang benar, tetapi maknanya tidak sesuai dengan aturan bahasa pemrograman. Dengan kata lain, program dapat lolos dari tahap lexical analysis dan syntax analysis, tetapi tetap dianggap salah karena terdapat pelanggaran aturan semantik. Semantic error berbeda dengan syntax error. Syntax error terjadi ketika susunan token tidak sesuai dengan grammar bahasa pemrograman, misalnya lupa menuliskan tanda titik koma atau salah menuliskan struktur **if**. Sementara itu, semantic error terjadi ketika struktur program sudah benar secara grammar, tetapi penggunaannya tidak bermakna benar. Contohnya adalah menggunakan variabel yang belum dideklarasikan, memberikan nilai bertipe **String** ke variabel bertipe **Integer**, atau memanggil fungsi dengan jumlah argumen yang tidak sesuai.

Salah satu bentuk semantic error yang umum adalah penggunaan identifier yang belum dideklarasikan. Dalam sebuah program, variabel, fungsi, atau prosedur biasanya harus dideklarasikan terlebih dahulu sebelum digunakan. Jika compiler menemukan identifier yang tidak tercatat di dalam symbol table, maka compiler dapat menghasilkan semantic error karena identifier tersebut tidak dikenali. Semantic error juga dapat terjadi akibat deklarasi ulang identifier pada scope yang sama. Misalnya, jika suatu variabel bernama **x** sudah dideklarasikan di dalam suatu blok, kemudian programmer mendeklarasikan kembali variabel **x** pada blok yang sama, maka hal tersebut dapat dianggap sebagai kesalahan. Deklarasi ulang dapat menyebabkan ambiguitas karena compiler tidak dapat menentukan deklarasi mana yang seharusnya digunakan.

Kesalahan tipe data juga merupakan salah satu bentuk semantic error. Contohnya adalah melakukan operasi aritmatika pada tipe data yang tidak sesuai, seperti menjumlahkan **Integer** dengan

`String`, atau menggunakan nilai `Integer` sebagai kondisi pada statement `if` yang seharusnya membutuhkan nilai Boolean. Kesalahan seperti ini dapat dideteksi melalui proses type checking dan type compatibility. Semantic error juga dapat muncul pada pemanggilan fungsi atau prosedur. Misalnya, sebuah fungsi dideklarasikan menerima dua parameter, tetapi saat dipanggil hanya diberikan satu argumen. Kesalahan juga dapat terjadi jika tipe argumen tidak sesuai dengan tipe parameter yang diharapkan. Dalam kasus seperti ini, struktur pemanggilan fungsi mungkin benar secara sintaksis, tetapi salah secara semantik. Selain itu, semantic error dapat terjadi pada penggunaan struktur data. Misalnya, mengakses elemen array menggunakan indeks dengan tipe yang tidak sesuai, mengakses field yang tidak terdapat pada suatu record, atau memperlakukan variabel biasa seperti array. Kesalahan-kesalahan tersebut menunjukkan bahwa program menggunakan suatu objek tidak sesuai dengan definisi atau tipenya.

BAB 2 Perancangan Program

2.1 Gambaran Umum dan Teknologi yang Digunakan

Pada milestone ketiga ini, program yang dikembangkan merupakan lanjutan dari program lexical analyzer pada milestone pertama dan syntax analysis pada milestone kedua. Pada milestone pertama, program berfokus untuk memecah source code menjadi token-token. Pada milestone kedua, program dilanjutkan dengan menambahkan bagian syntax analyzer atau parser untuk memeriksa kesesuaian susunan token terhadap grammar bahasa Arion. Pada milestone ketiga ini, program dilanjutkan dengan menambahkan tahap semantic analysis, yaitu proses untuk memeriksa makna dari program yang telah dibuat. Tahap ini bertujuan untuk memastikan bahwa program tidak hanya benar secara sintaksis, tetapi juga benar secara semantik.

Program ini umumnya bekerja dengan menerima masukan berupa file input yang berisi source code Arion. File tersebut akan diproses terlebih dahulu oleh lexer untuk menghasilkan daftar token. Setelah itu, daftar token tersebut akan diberikan kepada parser untuk diperiksa struktur atau urutannya. Jika susunan token sudah sesuai dengan aturan grammar yang digunakan, parser akan membentuk parse tree yang menunjukkan susunan program. Hasil parse tree tersebut kemudian akan dilanjutkan ke tahap pembentukan Abstract Syntax Tree atau AST. Pada tahap ini, informasi sintaksis yang tidak diperlukan akan disederhanakan sehingga struktur program dapat direpresentasikan dalam bentuk yang lebih ringkas dan lebih sesuai untuk proses semantic analysis.

Setelah AST terbentuk, program akan melakukan proses semantic analysis. Proses ini dilakukan dengan menelusuri node-node pada AST untuk memeriksa berbagai aturan semantik, seperti pengecekan deklarasi identifier, pengecekan lingkup atau scope, serta pengecekan kesesuaian tipe data pada ekspresi dan statement. Selain itu, program juga akan menggunakan symbol table untuk menyimpan informasi mengenai identifier yang ditemukan, seperti nama variabel, konstanta, fungsi, prosedur, tipe data, dan informasi lain yang dibutuhkan dalam proses analisis.

Hasil dari proses semantic analysis adalah Decorated Abstract Syntax Tree dan symbol table. Decorated AST merupakan AST yang telah diberi anotasi tambahan, seperti informasi tipe data, referensi ke symbol table, dan informasi scope. Jika ditemukan kesalahan semantik, program akan menampilkan pesan error yang menjelaskan jenis kesalahan yang terjadi. Dengan adanya tahap ini, program dapat memastikan bahwa source code Arion yang dianalisis tidak hanya sesuai dengan grammar, tetapi juga memiliki makna yang valid sesuai aturan bahasa.

Program ini dibuat menggunakan bahasa C++. Bahasa pemrograman ini dipakai karena program pada milestone sebelumnya juga dibuat menggunakan C++, sehingga pengembangan semantic analyzer dapat langsung melanjutkan struktur program yang sudah ada. Selain itu, C++ menyediakan struktur data seperti vector, string, map, stack, dan class yang dapat membantu dalam penyimpanan token, pembuatan node parse tree, pembentukan AST, pengelolaan symbol table, serta proses semantic analysis.

Program ini juga menggunakan Makefile untuk proses kompilasi supaya proses build dapat dilakukan dengan lebih mudah. Dengan adanya Makefile ini, file-file program dapat dikompilasi melalui satu perintah. Hal ini membuat proses menjalankan program menjadi lebih mudah dan praktis, terutama karena program terdiri dari beberapa modul seperti lexer, parser, AST, symbol table, dan semantic analyzer.

2.2 Struktur Program dan Fungsi/Kelas Utama

Untuk memastikan program mudah dikembangkan, dipahami, dan diuji, struktur program pada milestone ketiga ini dirancang secara modular. Setiap bagian program dipisahkan berdasarkan tanggung jawabnya masing-masing. Pemisahan ini bertujuan agar proses lexical analysis, syntax analysis, pembentukan AST, semantic analysis, dan pengelolaan symbol table dapat dilakukan secara lebih teratur.

Pada milestone ini, program tidak hanya terdiri dari modul lexer dan parser seperti pada milestone sebelumnya, tetapi juga ditambahkan modul semantic analyzer. Modul semantic analyzer bertugas untuk memeriksa makna dari program yang telah diproses oleh parser. Selain itu, terdapat juga struktur data tambahan untuk merepresentasikan AST, Decorated AST, dan symbol table yang digunakan selama proses semantic analysis berlangsung. Secara umum, struktur direktori program yang digunakan adalah sebagai berikut.

```
src/  
├── main.cpp  
├── lexer/  
│   ├── lexer.hpp  
│   ├── lexer.cpp  
│   ├── token.hpp  
│   └── token.cpp  
├── parser/  
│   ├── parser.hpp  
│   └── parser.cpp  
└── semantic_analysis/  
    ├── ast.hpp  
    ├── ast.cpp  
    ├── ast_printer.hpp  
    ├── ast_printer.cpp  
    ├── semantic_analyzer.hpp  
    ├── semantic_analyzer.cpp  
    ├── symbol_table.hpp  
    ├── symbol_table.cpp  
    └── type.hpp
```

Penjelasan dari masing-masing bagian direktori adalah sebagai berikut:

1. **src/** : Merupakan direktori utama yang menyimpan seluruh source code program.
2. **src/main.cpp** : Merupakan file utama yang mengatur alur eksekusi program. File ini menghubungkan proses pembacaan input, tokenisasi oleh lexer, parsing oleh parser, pembentukan AST, semantic analysis, dan pencetakan hasil akhir ke terminal serta file output.
3. **src/lexer/** : Merupakan folder yang berisi bagian lexical analyzer. Bagian ini digunakan untuk membaca source code Arion dan mengubahnya menjadi daftar token yang akan digunakan oleh parser.

4. **src/parser/** : Merupakan folder yang berisi bagian syntax analyzer atau parser. Parser digunakan untuk memeriksa susunan token berdasarkan grammar bahasa Arion dan membentuk parse tree.
5. **src/semantic_analysis/** : Merupakan folder yang berisikan bagian semantic analyzer. Bagian ini digunakan untuk memeriksa apakah semantic pada program sudah sesuai atau tidak. Folder ini berisi struktur AST, proses konversi parse tree menjadi AST, semantic analyzer, symbol table, serta printer untuk menampilkan Decorated AST dan symbol table.
6. **doc/** : Merupakan folder yang digunakan untuk menyimpan laporan.
7. **test/** : Merupakan folder yang digunakan untuk menyimpan file input dan output pengujian.
8. **Makefile** : Digunakan untuk membantu proses kompilasi program agar dapat dijalankan dengan lebih mudah melalui satu perintah.

2.2.1 Fungsi dan Modul Utama Program pada Semantic Analysis

Dalam proses semantic analisis program ini, terdapat beberapa fungsi dan kelas utama yang digunakan untuk menjalankan proses dari source code sampai menjadi parse tree dari hasil program parser sebelumnya.

1. Modul AST (**ast.hpp** dan **ast.cpp**) :

Modul ini digunakan untuk merepresentasikan Abstract Syntax Tree dan membangun AST dari parse tree.

- **class ASTNode** : Merupakan kelas dasar untuk seluruh node AST. Kelas ini menyimpan informasi umum seperti tipe node, daftar child, tipe semantik, indeks symbol table, dan lexical level.
- **Node turunan AST** : Program memiliki beberapa node turunan seperti ProgramNode, VarDeclNode, TypeDeclNode, ConstDeclNode, AssignNode, BinOpNode, VarNode, IfNode, WhileNode, ForNode, ProcCallNode, ArrayAccessNode, dan RecordAccessNode.
- **buildAST()** : Merupakan fungsi utama yang mengubah parse tree hasil parser menjadi AST. Fungsi ini memanggil beberapa helper seperti buildExpression(), buildStatement(), buildType(), dan buildDeclarationPart() untuk membentuk node AST yang lebih ringkas.

Selain menyimpan informasi umum node, setiap node AST juga memiliki fungsi **visit(SemanticAnalyzer& analyzer)**. Fungsi ini digunakan sebagai mekanisme visitor untuk meneruskan proses analisis ke fungsi visitor yang sesuai pada kelas **SemanticAnalyzer**. Sebagai contoh, **AssignNode::visit()** akan memanggil

`analyzer.visitAssign(this)`, sedangkan `IfNode::visit()` akan memanggil `analyzer.visitIf(this)`.

2. Modul Symbol Table (`symbol_table.hpp` dan `symbol_table.cpp`) :

Modul ini digunakan untuk menyimpan informasi identifier yang ditemukan selama proses semantic analysis.

- **struct SymbolEntry**: Menyimpan informasi simbol seperti nama, jenis simbol, tipe data, lexical level, indeks TAB, parameter, referensi, dan atribut lain yang dibutuhkan.
- **struct TabEntry, ATabEntry, dan BTabEntry**: Digunakan untuk merepresentasikan tiga tabel utama pada milestone ini, yaitu TAB, ATAB, dan BTAB.
- Fungsi penting pada SymbolTable. Beberapa fungsi utama adalah `declareSymbol()`, `lookup()`, `enterScope()`, `exitScope()`, `addArrayType()`, `addBlockEntry()`, `addRecordField()`, dan `printSpecTables()`.

3. Modul Semantic Analyzer(`semantic_analyzer.hpp` dan `semantic_analyzer.cpp`) :

Modul ini merupakan bagian utama pada pengecekan semantic pada program dengan melakukan pengecekan makna program berdasarkan AST yang telah dibentuk pada modul sebelumnya.

- **Class SemanticAnalyzer** : Kelas yang mengelola seluruh proses semantic analysis, termasuk pengecekan deklarasi, scope, type compatibility, fungsi/prosedur, array, record, subrange, dan enumerated type.
- **analyze()** : fungsi utama untuk memulai proses semantic analysis dari root AST.
- **analyzeProgram(), analyzeDeclaration(), dan analyzeStatement()** : Digunakan untuk menganalisis bagian program ,deklarasi, dan statement sesuai jenis node yang sedang diproses.
- **inferExpressionType()** : fungsi untuk menentukan tipe data dari suatu ekspresi dan juga membantu proses type checking pada operasi aritmatika, logika, relasional, assignment, dan pemanggilan fungsi.
- **isAssignmentCompatible()** dan **isComparisonCompatible()** : fungsi untuk memeriksa apakah dua tipe data kompatibel dalam assignment atau comparison.
- **printErrors()** dan **printSymbolTables** : fungsi untuk mencetak error semantik dan symbol table hasil analisis.
- **visitProgram(), visitVarDecl(), visitTypeDecl(), visitConstDecl(), visitAssign(), visitIf(), visitWhile(), visitFor(), visitProcCall(), visitFuncCall(), visitArrayAccess()**, dan fungsi visitor lainnya digunakan

sebagai fungsi penerima dari **visit()** pada masing-masing node AST. Fungsi-fungsi ini kemudian meneruskan proses analisis ke fungsi internal, seperti **analyzeProgram()**, **analyzeDeclaration()**, **analyzeStatement()**, atau **inferExpressionType()** sesuai jenis node yang sedang diproses.

4. Modul AST Printer(**ast_printer.hpp** dan **ast_printer.cpp**) :

Modul ini digunakan untuk menampilkan DecoratedAST dalam bentuk tree yang lebih mudah dibaca dan menampilkannya pada output dalam bentuk file.

- **Class ASTPrinter** : kelas yang berfungsi mencetak AST yang sudah diberi anotasi semantik pada program sebelumnya.
- **Printer()** : Fungsi utama untuk mencetak root AST ke output stream.
- Fungsi bantuan printer. Beberapa fungsi bantuan seperti **printNode()**, **printValue()**, **printType()**, dan **writeAnnotation()** digunakan untuk mencetak node sesuai jenisnya serta menampilkan anotasi seperti tipe data, indeks symbol table, dan lexical level.

2.3 Alur Kerja

Alur kerja program pada milestone ketiga ini dimulai dari pembacaan file input yang berisi source code Arion. File input diberikan oleh pengguna melalui command line argument, bersamaan dengan file output yang akan digunakan untuk menyimpan hasil akhir program. Pada tahap ini, program memastikan bahwa argumen input dan output telah diberikan. Jika argumen tidak lengkap, program akan menampilkan format penggunaan program. Setelah file input diterima, program menjalankan tahap lexical analysis dengan memanggil fungsi **tokenize()**. Fungsi ini membaca isi file input karakter demi karakter, kemudian mengubahnya menjadi daftar token. Token yang dihasilkan dapat berupa identifier, keyword, konstanta, operator, delimiter, maupun token lain yang dikenali oleh lexer. Hasil dari tahap ini berupa **vector<Token>** yang kemudian digunakan sebagai masukan untuk tahap parser. Tahap berikutnya adalah syntax analysis. Daftar token yang dihasilkan oleh lexer diberikan kepada objek Parser. Parser kemudian menjalankan proses parsing menggunakan pendekatan Recursive Descent. Setiap aturan grammar direpresentasikan dalam fungsi parser tertentu, seperti **Program()**, **DeclarationPart()**, **Statement()**, **Expression()**, dan fungsi lainnya. Jika susunan token sesuai dengan grammar bahasa Arion, parser akan membentuk parse tree. Jika terdapat susunan token yang tidak sesuai, parser akan menghasilkan syntax error.

Setelah parse tree berhasil dibentuk, program melanjutkan proses pembentukan Abstract Syntax Tree atau AST. Proses ini dilakukan dengan memanggil fungsi **buildAST()**. Fungsi tersebut menerima root dari parse tree, kemudian mengubahnya menjadi AST yang lebih ringkas. Pada tahap ini, informasi

sintaksis yang tidak diperlukan untuk analisis semantik, seperti beberapa keyword, tanda baca, atau node perantara tertentu, akan dihilangkan. Informasi penting seperti deklarasi, ekspresi, assignment, pemanggilan prosedur/fungsi, dan struktur kontrol tetap dipertahankan dalam bentuk node AST. AST yang telah terbentuk kemudian diberikan kepada SemanticAnalyzer melalui fungsi `analyze()`. Fungsi ini memvalidasi root AST dan memastikan bahwa root merupakan ProgramNode. Jika root valid, proses traversal dimulai dengan memanggil `program->visit(*this)`. Setelah itu, setiap node AST diproses melalui fungsi `visit(SemanticAnalyzer& analyzer)` milik node tersebut. Fungsi `visit()` akan meneruskan proses ke method visitor yang sesuai pada SemanticAnalyzer, seperti `visitProgram()`, `visitAssign()`, `visitIf()`, `visitWhile()`, atau `visitBinOp()`. Dengan mekanisme ini, traversal AST tetap mengikuti pola visitor, sedangkan state analisis seperti symbol table, type registry, scope, dan daftar error tetap dikelola secara terpusat oleh SemanticAnalyzer.

Pada tahap semantic analysis, program melakukan beberapa pengecekan utama, seperti memastikan identifier telah dideklarasikan sebelum digunakan, mendeteksi deklarasi ulang pada scope yang sama, memeriksa kompatibilitas tipe data, memvalidasi assignment, mengecek ekspresi boolean pada struktur kontrol, serta memeriksa penggunaan array, record, subrange, enumerated type, prosedur, dan fungsi. Selama proses semantic analysis, program juga mengelola symbol table. Symbol table digunakan untuk menyimpan informasi identifier seperti nama, jenis simbol, tipe data, lexical level, referensi, dan atribut lain yang dibutuhkan. Symbol table terdiri dari tiga tabel utama, yaitu TAB, ATAB, dan BTAB. TAB digunakan untuk menyimpan informasi identifier, ATAB digunakan untuk menyimpan informasi array, sedangkan BTAB digunakan untuk menyimpan informasi block, procedure, function, dan record. Selain itu, program juga menginisialisasi reserved words dan predefined identifiers seperti `true`, `false`, `read`, `readln`, `write`, dan `writeln`. Jika semantic analyzer menemukan kesalahan, program akan menyimpan pesan error tersebut dan menampilkannya pada output. Jika tidak ditemukan error, AST akan diberi anotasi berupa informasi semantik seperti tipe data, indeks symbol table, dan lexical level. AST yang telah diberi anotasi ini disebut Decorated AST.

Tahap terakhir adalah pencetakan hasil. Program menggunakan ASTPrinter untuk mencetak Decorated AST dalam bentuk tree yang mudah dibaca. Selain itu, program juga mencetak hasil symbol table yang berisi TAB, ATAB, dan BTAB. Output akhir program ditampilkan ke terminal dan disimpan ke file output yang telah diberikan oleh pengguna. Dengan alur ini, program pada milestone ketiga menghasilkan output semantic analysis yang berisi status pipeline, error semantik jika ada, Decorated AST, dan Symbol Table.

2.4 Rancangan AST dan Decorated AST

Parse tree yang dihasilkan oleh parser tidak langsung digunakan sebagai struktur utama untuk semantic analysis. Hal ini disebabkan karena parse tree masih menyimpan banyak informasi sintaksis yang tidak diperlukan pada tahap analisis semantik, seperti simbol terminal tertentu, tanda baca, keyword, dan node grammar perantara. Oleh karena itu, program membentuk Abstract Syntax Tree atau AST sebagai representasi yang lebih ringkas dari struktur program. AST dirancang untuk menyimpan bagian-bagian program yang memiliki makna semantik.

Proses pembentukan AST dilakukan oleh fungsi `buildAST()` yang menerima parse tree sebagai masukan. Fungsi tersebut kemudian memanggil beberapa fungsi bantuan seperti `buildDeclarationPart()`, `buildStatementList()`, `buildStatement()`, `buildExpression()`, `buildSimpleExpression()`, `buildTerm()`, `buildFactor()`, dan `buildType()`. Setiap fungsi bantuan tersebut bertanggung jawab untuk membaca bagian tertentu dari parse tree dan mengubahnya menjadi node AST yang sesuai. Sebagai contoh, bagian deklarasi pada parse tree akan diubah menjadi node deklarasi seperti `VarDeclNode`, `ConstDeclNode`, atau `TypeDeclNode`, sedangkan bagian statement akan diubah menjadi node seperti `AssignNode`, `IfNode`, `WhileNode`, `ForNode`, atau `ProcCallNode`.

Dalam rancangan ini, AST dibuat dengan menghilangkan detail sintaksis yang tidak lagi dibutuhkan, yaitu keyword seperti `begin`, `end`, `var`, atau `program` tidak disimpan sebagai node utama dalam AST karena keyword tersebut hanya dibutuhkan untuk proses parsing. Begitu juga dengan tanda baca seperti titik koma, koma, tanda kurung, dan operator delimiter tertentu. Informasi yang disimpan hanyalah informasi yang diperlukan untuk memahami makna program. Misalnya, pada assignment `x := a + b`, AST cukup menyimpan node assignment dengan target berupa variabel `x` dan nilai berupa ekspresi operasi penjumlahan antara `a` dan `b`. Setiap node AST diturunkan dari kelas dasar `ASTNode`. Node-node turunan kemudian dibuat sesuai kebutuhan struktur program, yaitu `ProgramNode` untuk program utama, `VarDeclNode` untuk deklarasi variabel, `ConstDeclNode` untuk deklarasi konstanta, `TypeDeclNode` untuk deklarasi tipe, serta `AssignNode` untuk statement assignment.

Untuk merepresentasikan nilai dan ekspresi, program menggunakan turunan dari `ValueNode`. Node yang termasuk dalam bagian ini antara lain `NumberNode`, `StringNode`, `CharNode`, `VarNode`, `UnaryOpNode`, `BinOpNode`, `FuncCallNode`, `ArrayAccessNode`, dan `RecordAccessNode`. Node-node tersebut digunakan untuk menyimpan literal, penggunaan variabel, operasi unary, operasi biner, pemanggilan fungsi, akses elemen array, dan akses field record. Pemisahan ini memudahkan semantic analyzer dalam menentukan tipe suatu ekspresi. Selain node ekspresi, AST juga memiliki node khusus untuk tipe data. Tipe data direpresentasikan menggunakan `TypeNode` dan beberapa turunannya,

seperti `RangeNode`, `ArrayTypeNode`, `RecordTypeNode`, `FieldPartNode`, dan `EnumeratedTypeNode`. Dengan struktur ini, tipe sederhana, subrange, array, record, dan enumerated type dapat dianalisis sesuai aturan masing-masing. Misalnya, `ArrayTypeNode` menyimpan informasi tipe indeks dan tipe elemen, sedangkan `RecordTypeNode` menyimpan daftar field yang dimiliki oleh record.

Setelah AST selesai dibentuk, program melakukan proses semantic analysis untuk menghasilkan Decorated AST. Decorated AST adalah AST yang telah diberi anotasi tambahan berdasarkan hasil analisis semantik. Anotasi tersebut disimpan pada atribut `semanticType`, `tabIndex`, dan `lexicalLevel` yang terdapat pada setiap `ASTNode`. Atribut `semanticType` digunakan untuk menyimpan tipe data hasil analisis suatu node. Sebagai contoh, node variabel akan diberi tipe sesuai deklarasinya, node literal integer akan diberi tipe integer, dan node operasi biner akan diberi tipe berdasarkan tipe operand yang digunakan. Atribut ini digunakan oleh semantic analyzer untuk memastikan bahwa operasi dan assignment dilakukan dengan tipe yang kompatibel.

Atribut `tabIndex` digunakan untuk menyimpan referensi node terhadap entri pada symbol table. Ketika semantic analyzer menemukan penggunaan identifier, analyzer akan mencari identifier tersebut di symbol table. Jika ditemukan, node yang bersangkutan akan diberi indeks symbol table yang sesuai. Dengan demikian, Decorated AST dapat menunjukkan hubungan antara penggunaan identifier dan deklarasinya. Atribut `lexicalLevel` digunakan untuk menunjukkan scope tempat node tersebut berada. Nilai ini membantu membedakan identifier yang berada pada scope global, prosedur, fungsi, atau blok tertentu. Informasi ini penting karena identifier dengan nama yang sama dapat muncul pada scope yang berbeda.

2.5 Rancangan Symbol Table

Symbol table digunakan untuk menyimpan informasi mengenai identifier yang muncul di dalam program. Identifier tersebut dapat berupa nama program, variabel, konstanta, tipe data, prosedur, fungsi, parameter, maupun field pada record. Symbol table direpresentasikan melalui kelas `SymbolTable` yang menyimpan beberapa struktur data utama, yaitu `scopes`, `tab`, `atab`, dan `btab`. Struktur `scopes` digunakan untuk mengelola lingkup atau scope identifier. Setiap scope menyimpan daftar identifier yang dideklarasikan pada lingkup tersebut. Ketika program memasuki scope baru, seperti procedure atau function, semantic analyzer akan memanggil fungsi `enterScope()`. Sebaliknya, ketika keluar dari scope tersebut, semantic analyzer akan memanggil fungsi `exitScope()`.

Tabel utama yang digunakan dalam rancangan ini terdiri dari tiga bagian, yaitu TAB, ATAB, dan BTAB. TAB digunakan untuk menyimpan informasi identifier secara umum. Setiap entri pada TAB direpresentasikan oleh TabEntry. Informasi yang disimpan pada TAB meliputi nama identifier, link ke identifier sebelumnya dalam scope yang sama, jenis objek, tipe data, referensi ke tabel lain, status normal parameter, lexical level, dan alamat atau offset. Dengan informasi tersebut, semantic analyzer dapat mengetahui jenis dan tipe suatu identifier ketika identifier tersebut digunakan. ATAB digunakan untuk menyimpan informasi tipe array. Setiap entri pada ATAB direpresentasikan oleh ATabEntry. Informasi yang disimpan meliputi tipe indeks, tipe elemen, referensi elemen, batas bawah indeks, batas atas indeks, ukuran elemen, dan ukuran total array. Tabel ini digunakan ketika program menemukan deklarasi array atau akses terhadap elemen array. Dengan adanya ATAB, semantic analyzer dapat memeriksa apakah indeks array memiliki tipe yang valid dan apakah elemen array digunakan sesuai tipe elemennya. BTAB digunakan untuk menyimpan informasi block. Setiap entri pada BTAB direpresentasikan oleh BTabEntry. Informasi yang disimpan meliputi indeks identifier terakhir dalam block, parameter terakhir, ukuran parameter, dan ukuran variabel lokal. Tabel ini digunakan untuk merepresentasikan block program, procedure, function, maupun record. Untuk record, BTAB juga digunakan untuk menyimpan daftar field yang dimiliki oleh record tersebut. Program juga menggunakan SymbolEntry sebagai struktur sementara untuk menyimpan informasi simbol sebelum dimasukkan ke dalam TAB. SymbolEntry menyimpan nama simbol, jenis simbol, tipe data, lexical level, indeks symbol table, daftar parameter, referensi, dan atribut tambahan lain yang diperlukan. Jenis simbol dibedakan menggunakan SymbolKind, seperti Program, Variable, Constant, Type, Function, Procedure, Parameter, Field, dan Reserved.

Pada awal proses semantic analysis, symbol table diinisialisasi dengan reserved words dan predefined identifiers. Reserved words seperti and, array, begin, case, const, div, downto, do, else, end, for, function, if, mod, not, of, or, procedure, program, record, repeat, integer, real, boolean, char, string, then, to, type, until, var, dan while dimasukkan terlebih dahulu. Selain itu, predefined identifiers seperti true, false, read, readln, write, dan writeln juga dimasukkan agar dapat dikenali pada proses semantic analysis. Proses penambahan identifier ke dalam symbol table dilakukan melalui fungsi declareSymbol(). Fungsi ini akan memeriksa apakah identifier dengan nama yang sama sudah ada pada scope aktif. Jika sudah ada, maka deklarasi tersebut dianggap tidak valid karena terjadi redeclaration pada scope yang sama. Jika belum ada, identifier akan dimasukkan ke dalam scope aktif dan juga dicatat ke dalam TAB. Dengan cara ini, program dapat mendeteksi deklarasi ulang secara langsung ketika semantic analyzer memproses bagian deklarasi. Untuk mencari identifier, program menggunakan fungsi lookup(). Fungsi ini mencari identifier mulai dari scope terdalam hingga scope terluar. Mekanisme ini sesuai dengan aturan scope resolution, yaitu identifier lokal akan diprioritaskan dibandingkan identifier pada scope luar. Jika identifier tidak ditemukan pada seluruh scope, maka semantic analyzer akan menghasilkan semantic error

karena identifier tersebut digunakan sebelum dideklarasikan. Symbol table juga memiliki fungsi tambahan untuk menangani tipe komposit. Fungsi `addArrayType()` digunakan untuk menambahkan informasi array ke dalam ATAB, sedangkan fungsi `addBlockEntry()` digunakan untuk menambahkan block baru ke dalam BTAB. Untuk record, fungsi `addRecordField()` digunakan untuk memasukkan field record ke dalam struktur block yang sesuai. Dengan adanya fungsi-fungsi ini, symbol table dapat menyimpan informasi tipe sederhana maupun tipe terstruktur.

2.6 Rancangan Semantic Checking

Setelah parse tree berhasil dibentuk oleh parser dan diubah menjadi AST, `SemanticAnalyzer` akan menelusuri AST tersebut menggunakan mekanisme visitor. Proses semantic checking dimulai dengan memanggil fungsi `analyze()` pada kelas `SemanticAnalyzer`. Fungsi ini menerima root AST, membersihkan daftar error, menginisialisasi type registry, serta memastikan bahwa root merupakan `ProgramNode`. Jika root valid, proses traversal dimulai dengan memanggil `program->visit(*this)`. Pemanggilan ini akan diteruskan ke `visitProgram()` pada `SemanticAnalyzer`, yang kemudian menjalankan analisis program utama melalui `analyzeProgram()`. Pada proses traversal berikutnya, setiap node diproses melalui fungsi `analyzeNode()`. Fungsi ini tidak lagi menjadi tempat utama untuk menentukan seluruh jenis node, tetapi memanggil `node->visit(*this)` agar pemilihan fungsi analisis awal dilakukan melalui mekanisme visitor. Dengan demikian, setiap node AST bertanggung jawab meneruskan dirinya ke fungsi visitor yang sesuai.

Pengecekan pertama yang dilakukan adalah pengecekan deklarasi identifier. Setiap deklarasi variabel, konstanta, tipe, prosedur, fungsi, dan parameter akan dimasukkan ke dalam symbol table. Proses ini dilakukan melalui fungsi `declareOrReport()`, yang memanggil fungsi `declareSymbol()` dari `SymbolTable`. Jika identifier dengan nama yang sama sudah terdapat pada scope aktif, maka semantic analyzer akan menghasilkan error karena terjadi deklarasi ulang pada scope yang sama. Selain mengecek deklarasi ulang, semantic analyzer juga melakukan pengecekan penggunaan identifier. Ketika suatu identifier digunakan pada ekspresi, assignment, pemanggilan prosedur, pemanggilan fungsi, akses array, atau akses record, analyzer akan mencari identifier tersebut di symbol table menggunakan fungsi `lookup()`. Jika identifier tidak ditemukan, maka program akan menghasilkan semantic error karena identifier digunakan sebelum dideklarasikan. Jika identifier ditemukan, node terkait akan diberi anotasi berupa tipe data, indeks symbol table, dan lexical level.

Pengecekan berikutnya adalah type checking. Type checking dilakukan untuk memastikan bahwa setiap ekspresi, assignment, operasi, dan pemanggilan fungsi menggunakan tipe data yang sesuai. Untuk menentukan tipe suatu ekspresi, semantic analyzer menggunakan fungsi `inferExpressionType()`. Fungsi ini akan menentukan tipe dari literal, variabel, operasi unary, operasi biner, pemanggilan fungsi, akses array, dan akses record. Hasil inferensi tipe tersebut kemudian digunakan untuk memeriksa apakah suatu

operasi valid atau tidak. Pada operasi aritmatika, semantic analyzer memastikan bahwa operand yang digunakan memiliki tipe numerik, yaitu integer atau real. Operasi seperti penjumlahan, pengurangan, dan perkalian hanya dapat dilakukan pada tipe numerik. Jika salah satu operand bukan tipe numerik, maka program akan menghasilkan semantic error. Untuk operasi pembagian real, hasil ekspresi akan diberi tipe real, sedangkan operasi aritmatika biasa akan menghasilkan integer apabila kedua operand bertipe integer. Pada operasi logika, semantic analyzer memastikan bahwa operand bertipe boolean. Operator seperti and, or, dan not hanya valid digunakan pada ekspresi boolean. Jika operand memiliki tipe selain boolean, maka analyzer akan menghasilkan error. Sementara itu, pada operasi relasional, semantic analyzer akan memeriksa kompatibilitas antara operand kiri dan operand kanan. Jika kedua tipe kompatibel, hasil ekspresi relasional akan diberi tipe boolean.

Pengecekan assignment dilakukan dengan membandingkan tipe variabel tujuan dan tipe ekspresi di sebelah kanan. Proses ini menggunakan fungsi `isAssignmentCompatible()`. Assignment dianggap valid apabila tipe tujuan dan tipe nilai kompatibel. Sebagai contoh, nilai bertipe integer dapat diberikan ke variabel bertipe integer, dan nilai bertipe integer juga dapat diberikan ke variabel bertipe real. Namun, nilai bertipe string tidak dapat diberikan ke variabel bertipe integer.

Semantic analyzer juga memeriksa struktur kontrol seperti if, while, dan repeat. Ekspresi kondisi pada struktur kontrol harus menghasilkan tipe boolean. Jika kondisi tidak bertipe boolean, maka program akan menghasilkan semantic error. Untuk for, analyzer memeriksa variabel kontrol, ekspresi awal, dan ekspresi akhir agar tipe yang digunakan sesuai. Statement di dalam blok kontrol juga akan dianalisis secara rekursif. Untuk tipe data terstruktur, semantic analyzer melakukan pengecekan khusus. Pada array, analyzer memastikan bahwa variabel yang diakses memang bertipe array dan indeks yang digunakan memiliki tipe yang sesuai dengan tipe indeks array. Pada record, analyzer memastikan bahwa variabel yang diakses memang bertipe record dan field yang digunakan terdapat di dalam definisi record tersebut. Jika field tidak ditemukan atau variabel bukan record, maka program akan menghasilkan semantic error. Pada subrange, semantic analyzer memeriksa batas bawah dan batas atas. Nilai batas bawah tidak boleh lebih besar dari batas atas, dan tipe subrange tidak boleh menggunakan tipe real. Selain itu, ketika suatu nilai diberikan ke variabel bertipe subrange, analyzer dapat memeriksa apakah nilai tersebut berada dalam batas yang valid. Pada enumerated type, analyzer memeriksa bahwa setiap member enumerated telah dideklarasikan sebelumnya dan seluruh member memiliki tipe yang sama. Jika terdapat member yang belum dideklarasikan, member duplikat, atau member dengan tipe berbeda, maka analyzer akan menghasilkan semantic error. Pengecekan ini dilakukan agar enumerated type tetap konsisten sesuai aturan semantik yang digunakan.

Semantic analyzer juga melakukan pengecekan terhadap pemanggilan prosedur dan fungsi. Ketika prosedur atau fungsi dipanggil, analyzer akan memastikan bahwa identifier tersebut memang merupakan prosedur atau fungsi. Selain itu, jumlah argumen dan tipe argumen akan dibandingkan dengan daftar parameter yang tersimpan pada symbol table. Jika jumlah argumen tidak sesuai atau tipe argumen tidak kompatibel, maka program akan menghasilkan semantic error. Untuk prosedur bawaan seperti read, readln, write, dan writeln, analyzer memperlakukan pemanggilan tersebut sebagai prosedur variadic sehingga jumlah argumennya lebih fleksibel. Selama proses semantic checking, setiap error yang ditemukan akan disimpan ke dalam daftar error. Program tidak langsung berhenti pada error pertama, tetapi berusaha melanjutkan analisis agar beberapa error dapat dilaporkan sekaligus. Setelah seluruh AST dianalisis, program akan memeriksa apakah terdapat error. Jika ada, hasil output akan menampilkan bagian Semantic Errors. Jika tidak ada, output akan menunjukkan bahwa semantic analyzer berhasil dan Decorated AST akan dicetak bersama symbol table.

BAB 3 Implementasi

3.1 Implementasi Program Pembentukan AST

Program pembentukan AST diimplementasikan melalui fungsi `buildAST()`. Fungsi ini menerima root parse tree hasil parser, kemudian mengubah struktur parse tree menjadi AST yang lebih ringkas. Program mengambil nama program dari `<program-header>`, membangun bagian deklarasi melalui `buildDeclarationPart()`, dan membangun isi program melalui `buildStatementList()`. Hasil akhirnya disimpan pada node `ProgramNode`.

```
ASTNode* buildAST(TreeParser* root){
    jika root kosong atau bukan <program>
        return null

    ambil nama program
    buat ProgramNode

    bangun declaration-part
    bangun statement-list

    tambahkan ke ProgramNode
    return ProgramNode
}
```

3.2 Implementasi Node Pencarian Child

Fungsi `findChild()` digunakan untuk mencari child tertentu berdasarkan nama node parse tree. Fungsi melakukan iterasi terhadap seluruh child dan mengembalikan node yang sesuai. Fungsi ini digunakan pada proses pembentukan AST seperti parameter list, field list, dan case block.

```
findChild(node, nama):
    untuk setiap child pada node
        jika data child = nama
            return child

    return null
```

3.3 Implementasi Node Variabel

Fungsi `buildVariable()` membentuk AST untuk variabel. Variabel dapat berupa variabel biasa, akses array, maupun akses record. Fungsi akan mengecek struktur child untuk menentukan tipe akses yang digunakan. Jika menggunakan indeks array maka dibuat `ArrayAccessNode`, sedangkan akses record menghasilkan `RecordAccessNode`.


```
buildVariable(node):  
    ambil nama variabel  
  
    jika hanya satu child  
        return VarNode  
  
    jika memakai []  
        return ArrayAccessNode  
  
    jika memakai .  
        return RecordAccessNode  
  
    return VarNode
```

3.4 Implementasi Node Factor

Fungsi `buildFactor()` membentuk AST untuk node `<factor>`. Factor dapat berupa literal, variabel, unary operator not, expression dalam kurung, maupun pemanggilan fungsi. Setiap bentuk factor akan dikonversi menjadi node AST sesuai jenisnya.

```
buildFactor(node):  
    jika not  
        return UnaryOpNode  
  
    jika (expression)  
        return buildExpression()  
  
    jika function call  
        return FuncCallNode  
  
    jika variable  
        return buildVariable()  
  
    jika literal  
        return node literal
```

3.5 Implementasi Node Term

Fungsi `buildTerm()` membentuk operasi perkalian, pembagian, div, mod, dan and. Fungsi membaca factor pertama kemudian menggabungkan factor berikutnya menggunakan `BinOpNode`.

```
buildTerm(node):  
    hasil = factor pertama  
  
    selama masih ada operator  
        ambil operator  
        ambil factor berikutnya
```

```
    hasil = BinOpNode  
  
    return hasil
```

3.6 Implementasi Node Simple Expression

Fungsi `buildSimpleExpression()` membentuk ekspresi penjumlahan, pengurangan, dan operator `or`. Fungsi juga mendukung unary plus dan minus pada awal ekspresi.

```
buildSimpleExpression(node):  
    cek unary + atau -  
  
    hasil = term pertama  
  
    selama ada operator aditif  
        buat BinOpNode  
  
    return hasil
```

3.7 Implementasi Node Expression

Fungsi `buildExpression()` membangun ekspresi relasional. Jika terdapat operator relasi maka dibuat `BinOpNode`, sedangkan jika hanya satu simple-expression maka hasil langsung dikembalikan.

```
buildExpression(node):  
    left = simpleExpression  
  
    jika ada operator relasi  
        right = simpleExpression  
        return BinOpNode  
  
    return left
```

3.8 Implementasi Node Type

Fungsi `buildType()` membentuk AST tipe data. Fungsi mendukung tipe array, record, enumerated, subrange, dan identifier biasa. Setiap jenis akan menghasilkan node turunan yang berbeda seperti `ArrayTypeNode`, `RecordTypeNode`, dan `RangeNode`.

```
buildType(node):  
    jika array  
        return ArrayTypeNode
```

```
jika record
    return RecordTypeNode

jika enumerated
    return EnumeratedTypeNode

jika range
    return RangeNode

return TypeNode
```

3.9 Implementasi Node Constant

Fungsi `buildConstant()` digunakan untuk membangun AST konstanta. Konstanta dapat berupa karakter, string, integer, real, maupun identifier. Fungsi juga mendukung unary plus dan minus.

```
buildConstant(node):
    cek unary + atau -

    jika char
        return CharNode

    jika string
        return StringNode

    jika angka
        return NumberNode

    return VarNode
```

3.10 Implementasi Parameter Formal

Fungsi `buildFormalParams()` membentuk parameter procedure/function dari `<formal-parameter-list>`. Setiap parameter disimpan sebagai `ParameterNode`. Fungsi juga mendukung parameter array dan parameter pass-by-reference (`var`).

```
buildFormalParams(node):
    untuk setiap parameter-group
        ambil identifier
        ambil tipe

        buat ParameterNode

    return parameter
```

3.11 Implementasi Statement

Fungsi `buildStatement()` digunakan untuk membentuk AST dari node `<statement>`. Fungsi akan memeriksa jenis statement yang ditemukan pada parse tree, seperti assignment, pemanggilan procedure/function, percabangan, perulangan, dan compound statement. Setiap jenis statement akan diproses oleh node AST yang sesuai. Jika statement kosong, fungsi akan mengembalikan `nullptr`.

```
buildStatement(node):  
    jika statement kosong  
        return null  
  
    ambil child pertama  
  
    jika assignment  
        return AssignNode  
  
    jika procedure/function call  
        return ProcCallNode  
  
    jika if statement  
        return IfNode  
  
    jika while statement  
        return WhileNode  
  
    jika repeat statement  
        return RepeatNode  
  
    jika for statement  
        return ForNode  
  
    jika case statement  
        return CaseNode  
  
    jika compound statement  
        return buildStatementList()
```

3.11.1 Implementasi Node Assignment Statement

Assignment statement diimplementasikan dengan membentuk `AssignNode`. Fungsi mengambil target variabel dan nilai ekspresi yang akan diberikan. Variabel tujuan direpresentasikan sebagai `VarNode`, sedangkan nilai dihasilkan melalui `buildExpression()`.

```
jika assignment statement:  
    ambil nama variabel
```

```
target ← VarNode
value ← buildExpression()

return AssignNode(target, value)
```

3.11.2 Implementasi Node Procedure/Function Call

Procedure atau function call dibentuk menjadi ProcCallNode. Fungsi mengambil nama procedure/function kemudian memproses seluruh parameter melalui `buildExpression()`.

```
jika procedure call:
    ambil nama procedure

    buat ProcCallNode

    untuk setiap parameter
        bangun expression
        tambahkan parameter

    return ProcCallNode
```

3.11.3 Implementasi Node If Statement

If statement direpresentasikan menggunakan IfNode. Fungsi membentuk kondisi menggunakan `buildExpression()`, kemudian membangun blok `then` dan `else` apabila tersedia.

```
jika if statement:
    condition ← buildExpression()

    thenBlock ← buildStatement()

    jika ada else
        elseBlock ← buildStatement()

    return IfNode
```

3.11.4 Implementasi Node While Statement

While statement dibentuk menjadi WhileNode. Fungsi membangun kondisi dan statement yang dieksekusi selama kondisi bernilai benar.

```
jika while statement:
    condition ← buildExpression()

    statement ← buildStatement()

    return WhileNode
```

3.11.5 Implementasi Node Repeat Statement

Repeat statement direpresentasikan menggunakan RepeatNode. Fungsi membangun blok statement melalui buildStatementList() dan kondisi penghentian melalui buildExpression().

```
jika repeat statement:
    statementList ← buildStatementList()

    condition ← buildExpression()

    return RepeatNode
```

3.11.6 Implementasi Node For Statement

For statement diimplementasikan menggunakan ForNode. Fungsi membentuk assignment awal iterator, nilai akhir iterasi, serta statement yang akan dijalankan.

```
jika for statement:
    ambil variabel iterator

    assignment ← AssignNode

    batas ← buildExpression()

    statement ← buildStatement()

    return ForNode
```

3.11.7 Implementasi Node Case Statement

Case statement dibentuk menggunakan CaseNode. Fungsi membangun ekspresi kondisi dan mengumpulkan seluruh CaseBlockNode secara rekursif.

```
jika case statement:
    condition ← buildExpression()

    buat CaseNode

    kumpulkan seluruh case-block:
        ambil constant
        ambil statement

        buat CaseBlockNode

    return CaseNode
```

3.12 Implementasi Statement List

Fungsi `buildStatementList()` membangun kumpulan statement. Seluruh statement disimpan pada node container `ProcCallNode("__block__")`. Setiap statement diproses melalui `buildStatement()`.

```
buildStatementList(node):  
    buat block  
  
    untuk setiap statement  
        bangun statement  
        tambahkan ke block  
  
    return block
```

3.13 Implementasi Declaration Part

Fungsi `buildDeclarations()` digunakan untuk membentuk AST pada bagian deklarasi program. Fungsi memproses seluruh deklarasi seperti konstanta, tipe data, variabel, procedure, dan function. Seluruh hasil deklarasi disimpan dalam container `__declarations__`.

```
buildDeclarations(node):  
    buat container deklarasi  
  
    untuk setiap child:  
        jika const declaration  
            proses konstanta  
  
        jika type declaration  
            proses tipe  
  
        jika var declaration  
            proses variabel  
  
        jika subprogram declaration  
            proses procedure/function  
  
    return container
```

3.13.1 Implementasi Node Const Declaration

Const declaration membentuk `ConstDeclNode`. Fungsi mengambil nama konstanta dan nilai yang bersesuaian, kemudian membentuk node berdasarkan tipe data konstanta.

```
untuk setiap deklarasi konstanta:  
    ambil nama konstanta
```

```
    bangun nilai konstanta  
  
    buat ConstDeclNode
```

3.13.2 Implementasi Node Type Declaration

Type declaration dibentuk menggunakan `TypeDeclNode`. Fungsi mengambil nama tipe dan memproses definisi tipenya menggunakan `buildType()`.

```
untuk setiap deklarasi tipe:  
    ambil nama tipe  
  
    type ← buildType()  
  
    buat TypeDeclNode
```

3.13.3 Implementasi Node Variable Declaration

Variable declaration diimplementasikan menggunakan `VarDeclNode`. Fungsi memproses daftar identifier dan tipe data yang digunakan.

```
untuk setiap identifier:  
    ambil tipe variabel  
  
    buat VarDeclNode
```

3.13.4 Implementasi Node Procedure Declaration

Procedure declaration direpresentasikan menggunakan `ProcDeclNode`. Fungsi membentuk parameter formal, deklarasi lokal, dan body procedure.

```
jika procedure declaration:  
    ambil nama procedure  
  
    buat ProcDeclNode  
  
    proses parameter  
  
    proses declaration part  
  
    proses statement body  
  
    return ProcDeclNode
```


3.13.5 Implementasi Node Function Declaration

Function declaration dibangun menggunakan `FuncDeclNode`. Selain parameter dan body, fungsi juga membentuk tipe nilai balik.

```
jika function declaration:
    ambil nama function

    ambil return type

    buat FuncDeclNode

    proses parameter

    proses declaration part

    proses statement body

    return FuncDeclNode
```

3.14 Implementasi Symbol Table

Symbol table diimplementasikan melalui kelas `SymbolTable`. Struktur ini digunakan untuk menyimpan informasi identifier yang ditemukan selama proses semantic analysis, seperti variabel, konstanta, tipe data, procedure, function, parameter, field, dan reserved word. Symbol table juga digunakan untuk melakukan pengecekan deklarasi ganda, pencarian identifier berdasarkan scope, serta pencetakan tabel simbol sesuai spesifikasi.

Pada implementasi ini, `SymbolTable` menggunakan struktur scope untuk mendukung pencarian identifier dari scope terdalam ke scope terluar. Selain itu, symbol table juga menyimpan struktur TAB, ATAB, dan BTAB agar sesuai dengan rancangan symbol table pada spesifikasi.

```
SymbolTable:
    buat scope global

    inisialisasi predefined identifier
    inisialisasi reserved word

    simpan data:
        scopes
        TAB
        ATAB
        BTAB
```

3.14.1 Implementasi Deklarasi Symbol

Deklarasi symbol dilakukan menggunakan fungsi `declareSymbol()`. Fungsi ini menerima data symbol, kemudian memeriksa apakah nama identifier sudah pernah dideklarasikan pada scope aktif. Jika

sudah ada, deklarasi dianggap tidak valid. Jika belum ada, symbol akan dimasukkan ke scope aktif dan dicatat ke dalam TAB.

```
declareSymbol(entry):  
    jika nama symbol sudah ada pada scope aktif  
        return false  
  
    isi lexical level sesuai scope saat ini  
    masukkan entry ke scope aktif  
    masukkan entry ke TAB  
    return true
```

3.14.2 Implementasi Scope

Scope digunakan untuk membedakan identifier yang berada pada lingkup berbeda. Ketika semantic analyzer memasuki procedure atau function, fungsi enterScope() akan dipanggil untuk membuat scope baru. Ketika proses analisis keluar dari procedure atau function, fungsi exitScope() akan dipanggil untuk menghapus scope aktif.

```
enterScope():  
    buat scope baru  
    naikan lexical level  
  
exitScope():  
    hapus scope aktif  
    turunkan lexical level
```

3.14.3 Implementasi Lookup Symbol

Scope digunakan untuk membedakan identifier yang berada pada lingkup berbeda. Ketika semantic analyzer memasuki procedure atau function, fungsi enterScope() akan dipanggil untuk membuat scope baru. Ketika proses analisis keluar dari procedure atau function, fungsi exitScope() akan dipanggil untuk menghapus scope aktif.

```
lookup(nama):  
    mulai dari scope terdalam  
  
    selama masih ada scope:  
        jika nama ditemukan  
            return symbol  
  
    pindah ke scope luar  
  
    return null
```

3.14.4 Implementasi TAB, ATAB, dan BTAB

Selain scope, symbol table juga menyimpan tiga tabel utama, yaitu TAB, ATAB, dan BTAB. TAB digunakan untuk menyimpan informasi identifier, ATAB digunakan untuk menyimpan informasi tipe array, sedangkan BTAB digunakan untuk menyimpan informasi block, procedure, function, dan record.

```
TAB:
    simpan identifier
    simpan jenis object
    simpan tipe data
    simpan lexical level
    simpan ref dan adr

ATAB:
    simpan tipe indeks array
    simpan tipe elemen array
    simpan batas bawah dan batas atas

BTAB:
    simpan informasi block
    simpan parameter dan variabel lokal
```

3.15 Implementasi Semantic Analyzer

Semantic analyzer diimplementasikan melalui kelas `SemanticAnalyzer`. Fungsi utama yang digunakan untuk memulai proses analisis adalah `analyze()`. Fungsi ini terlebih dahulu menghapus daftar error lama, membersihkan type registry, dan menginisialisasi tipe bawaan. Setelah itu, fungsi memeriksa apakah root AST kosong dan memastikan bahwa root merupakan `ProgramNode`. Jika root valid, proses analisis tidak langsung memanggil `analyzeProgram()`, melainkan dimulai melalui mekanisme visitor dengan memanggil `program->visit(*this)`. Pemanggilan tersebut akan mengarah ke `SemanticAnalyzer::visitProgram()`, lalu diteruskan ke `analyzeProgram()`.

Proses semantic analysis dimulai dari fungsi `analyze()`. Fungsi ini menerima root AST, kemudian memanggil fungsi analisis sesuai jenis node yang sedang diproses.

```
analyze(root):
    bersihkan daftar error
    bersihkan type registry
    inisialisasi builtin type

    jika root kosong:
        catat error
        return

    jika root bukan ProgramNode:
        catat error
        return

    program.visit(SemanticAnalyzer)
```

3.15.1 Implementasi Visit Function

Visit function digunakan untuk menghubungkan setiap node AST dengan fungsi analisis yang sesuai pada SemanticAnalyzer. Setiap node memiliki method visit(SemanticAnalyzer& analyzer). Method ini tidak menyimpan seluruh logika analisis secara langsung di dalam node, melainkan mendelegasikan analisis ke method visitor pada SemanticAnalyzer. Sebagai contoh, ProgramNode::visit() memanggil analyzer.visitProgram(this), AssignNode::visit() memanggil analyzer.visitAssign(this), dan BinOpNode::visit() memanggil analyzer.visitBinOp(this). Pendekatan ini membuat struktur AST tetap sederhana, sementara state penting seperti symbol table, scope, type registry, dan daftar error tetap dikelola secara terpusat.

```
ProgramNode.visit(analyzer):  
    analyzer.visitProgram(this)
```

```
AssignNode.visit(analyzer):  
    analyzer.visitAssign(this)
```

```
IfNode.visit(analyzer):  
    analyzer.visitIf(this)
```

```
BinOpNode.visit(analyzer):  
    analyzer.visitBinOp(this)
```

3.15.2 Implementasi Analisis Program

Fungsi analyzeProgram() digunakan untuk menganalisis program utama. Fungsi ini mendaftarkan nama program ke symbol table, kemudian memproses bagian deklarasi dan statement yang terdapat dalam program.

```
analyzeProgram(program):  
    daftarkan nama program ke symbol table  
  
    untuk setiap child pada program:  
        jika child adalah deklarasi  
            analyzeDeclaration(child)  
  
        jika child adalah statement  
            analyzeStatement(child)
```

3.15.3 Implementasi Analisis Deklarasi

Fungsi analyzeDeclaration() digunakan untuk memproses deklarasi pada AST. Deklarasi yang diproses meliputi deklarasi variabel, konstanta, tipe data, procedure, function, dan parameter. Setiap deklarasi akan dimasukkan ke dalam symbol table.

```
analyzeDeclaration(node):  
    jika VarDeclNode  
        masukkan variabel ke symbol table  
  
    jika ConstDeclNode  
        masukkan konstanta ke symbol table  
  
    jika TypeDeclNode  
        daftarkan tipe  
        masukkan tipe ke symbol table  
  
    jika ProcDeclNode  
        proses procedure  
  
    jika FuncDeclNode  
        proses function
```

3.15.4 Implementasi Analisis Statement

Fungsi `analyzeStatement()` digunakan untuk menganalisis statement pada AST. Fungsi ini memeriksa berbagai jenis statement seperti assignment, pemanggilan procedure, percabangan, perulangan, case, akses array, dan akses record.

```
analyzeStatement(node):  
    jika AssignNode  
        cek assignment  
  
    jika ProcCallNode  
        cek pemanggilan procedure  
  
    jika IfNode  
        cek kondisi dan statement cabang  
  
    jika WhileNode  
        cek kondisi dan body  
  
    jika RepeatNode  
        cek body dan kondisi until  
  
    jika ForNode  
        cek iterator dan batas perulangan  
  
    jika CaseNode  
        cek ekspresi case dan case block
```

3.15.5 Implementasi Inferensi Tipe Ekspresi

Fungsi `inferExpressionType()` digunakan untuk menentukan tipe dari suatu ekspresi. Fungsi ini memproses literal, variabel, operasi unary, operasi biner, pemanggilan function, akses array, dan akses record. Hasil tipe dari fungsi ini digunakan untuk proses type checking.

```
inferExpressionType(expr):  
    jika NumberNode  
        tentukan integer atau real  
  
    jika StringNode  
        return string  
  
    jika CharNode  
        return char  
  
    jika VarNode  
        cari tipe dari symbol table  
  
    jika UnaryOpNode  
        cek operator unary  
  
    jika BinOpNode  
        cek operator dan operand  
  
    jika FuncCallNode  
        cek function call  
        return tipe return function  
  
    jika ArrayAccessNode  
        return tipe elemen array  
  
    jika RecordAccessNode  
        return tipe field record
```

3.16 Implementasi Pengecekan Semantik

Pengecekan semantik dilakukan untuk memastikan bahwa program tidak hanya benar secara sintaksis, tetapi juga benar secara makna. Pengecekan ini dilakukan dengan menelusuri AST dan memanfaatkan symbol table untuk mengetahui informasi identifier, tipe data, scope, parameter, serta struktur data yang digunakan.

3.16.1 Implementasi Pengecekan Deklarasi Ganda

Deklarasi ganda dicek ketika identifier dimasukkan ke dalam symbol table. Jika identifier dengan nama yang sama sudah ada pada scope aktif, maka semantic analyzer akan menghasilkan error. Identifier dengan nama sama masih dapat digunakan pada scope berbeda karena shadowing diperbolehkan.

```
cekDeklarasi(entry):  
    jika entry sudah ada pada scope aktif  
        catat error deklarasi ganda  
  
    jika belum ada  
        masukkan entry ke symbol table
```

3.16.2 Implementasi Pengecekan Identifier

Setiap identifier yang digunakan dalam program akan dicari pada symbol table. Jika identifier tidak ditemukan, maka identifier tersebut dianggap belum dideklarasikan. Pengecekan ini dilakukan pada variabel, ekspresi, assignment, paman

giliran procedure/function, akses array, dan akses record.

```
cekIdentifier(nama):  
    symbol ← lookup(nama)  
  
    jika symbol kosong  
        catat error identifier belum dideklarasikan  
  
    return symbol
```

3.16.3 Implementasi Pengecekan Assignment

Pengecekan assignment dilakukan dengan membandingkan tipe variabel tujuan dan tipe ekspresi yang diberikan. Assignment dianggap valid jika kedua tipe sama atau jika variabel bertipe real menerima nilai bertipe integer. Jika tipe tidak kompatibel, semantic analyzer akan menghasilkan error.

```
cekAssignment(node):  
    targetType ← tipe variabel tujuan  
  
    valueType ← inferExpressionType(value)  
  
    jika targetType kompatibel dengan valueType  
        assignment valid  
    jika tidak kompatibel  
        catat error tipe assignment
```

3.16.4 Implementasi Pengecekan Operator

Operator dicek berdasarkan tipe operand yang digunakan. Operator aritmatika membutuhkan operand numerik, operator logika membutuhkan operand boolean, dan operator relasional menghasilkan nilai boolean. Jika operand tidak sesuai dengan operator, semantic analyzer akan menghasilkan error.

```
cekOperator(operator, leftType, rightType):  
    jika operator aritmatika  
        operand harus integer atau real  
  
    jika operator div atau mod  
        operand harus integer  
  
    jika operator logika  
        operand harus boolean  
  
    jika operator relasional  
        operand harus kompatibel  
        hasil boolean
```

3.16.5 Implementasi Pengecekan Struktur Kontrol

Struktur kontrol seperti if, while, dan repeat membutuhkan kondisi bertipe boolean. Untuk statement for, variabel iterator, nilai awal, dan nilai akhir harus bertipe integer. Jika aturan tersebut tidak terpenuhi, semantic analyzer akan menghasilkan error.

```
cekStrukturKontrol(node):  
    jika IfNode  
        kondisi harus boolean  
  
    jika WhileNode  
        kondisi harus boolean  
  
    jika RepeatNode  
        kondisi until harus boolean  
  
    jika ForNode  
        iterator harus integer  
        nilai awal harus integer  
        nilai akhir harus integer
```

3.16.6 Implementasi Scope Procedure dan Function

Procedure dan function dianalisis dengan membuat scope baru. Parameter dan deklarasi lokal dimasukkan ke scope tersebut. Setelah body procedure atau function selesai dianalisis, scope akan ditutup kembali. Untuk function, nama function juga dimasukkan ke scope lokal agar dapat digunakan sebagai variabel return.

```
analyzeProcedure(proc):  
    daftarkan procedure ke scope luar  
  
    enterScope()
```



```

    daftarkan parameter

    analisis deklarasi lokal

    analisis body procedure

    exitScope()

analyzeFunction(func):
    daftarkan function ke scope luar

    enterScope()

    daftarkan nama function sebagai return variable

    daftarkan parameter

    analisis deklarasi lokal

    analisis body function

    exitScope()

```

3.16.7 Implementasi Pengecekan Pemanggilan Procedure dan Function

Pemanggilan procedure dan function dicek berdasarkan symbol yang tersimpan pada symbol table. Semantic analyzer memastikan bahwa identifier yang dipanggil merupakan procedure atau function. Selain itu, jumlah argument dan tipe argument juga dibandingkan dengan parameter yang dideklarasikan.

```

cekCall(nama, arguments):
    symbol ← lookup(nama)

    jika symbol tidak ditemukan
        catat error

    jika symbol bukan procedure/function
        catat error

    jika procedure bawaan variadic
        return valid

    jika jumlah argument tidak sama dengan jumlah parameter
        catat error

    untuk setiap argument:
        argType ← inferExpressionType(argument)
        paramType ← tipe parameter

        jika argType tidak kompatibel dengan paramType

```

catat error

3.16.8 Implementasi Pengecekan Array

Pada akses array, semantic analyzer memeriksa apakah identifier yang diakses merupakan array. Setelah itu, semantic analyzer memeriksa tipe indeks yang digunakan. Jika indeks berupa literal dan batas array diketahui, analyzer juga memeriksa apakah indeks berada dalam batas yang valid.

```
cekArrayAccess(node):  
    cari symbol array  
  
    jika symbol bukan array  
        catat error  
  
    indexType ← inferExpressionType(index)  
  
    jika indexType tidak sesuai  
        catat error  
  
    jika indeks literal di luar batas  
        catat error  
  
    return tipe elemen array
```

3.16.9 Implementasi Pengecekan Record

Pada akses record, semantic analyzer memeriksa apakah identifier yang diakses merupakan record. Jika identifier bertipe record, analyzer akan memeriksa apakah field yang digunakan terdapat pada definisi record tersebut. Jika field tidak ditemukan, semantic analyzer akan menghasilkan error.

```
cekRecordAccess(node):  
    cari symbol record  
  
    jika symbol bukan record  
        catat error  
  
    jika field tidak ditemukan  
        catat error  
  
    return tipe field
```

3.16.10 Implementasi Pengecekan Subrange dan Case

Subrange dicek dengan memastikan batas bawah tidak lebih besar dari batas atas dan tipe batas tidak menggunakan real. Jika nilai literal diberikan ke variabel subrange, semantic analyzer dapat

memeriksa apakah nilai tersebut masih berada pada rentang yang valid. Pada case statement, tipe selector akan dibandingkan dengan tipe label case agar tetap kompatibel.

```
cekSubrange(range):  
    low ← batas bawah  
  
    high ← batas atas  
  
    jika low > high  
        catat error  
  
    jika tipe batas adalah real  
        catat error  
cekCase(node):  
    selectorType ← inferExpressionType(selector)  
  
    untuk setiap label case:  
        labelType ← inferExpressionType(label)  
  
        jika labelType tidak kompatibel dengan selectorType  
            catat error
```

3.17 Implementasi Output Semantic Analysis

Output semantic analysis digunakan untuk menampilkan hasil proses semantic analyzer. Jika ditemukan semantic error, program akan mencetak daftar error yang ditemukan. Jika tidak ada error, program akan menampilkan bahwa semantic analysis berhasil. Setelah itu, program juga mencetak Decorated AST dan symbol table.

```
outputSemanticAnalysis():  
    jika terdapat semantic error  
        cetak "=== Semantic Errors ==="  
        cetak seluruh error  
  
    jika tidak terdapat semantic error  
        cetak "Semantic analysis completed successfully"  
  
    cetak Decorated AST  
  
    cetak Symbol Tables
```

3.17.1 Implementasi Output Decorated AST

Decorated AST dicetak menggunakan ASTPrinter. Output ini menampilkan struktur AST yang sudah diberi anotasi hasil semantic analysis. Anotasi yang ditampilkan meliputi tipe semantik, indeks symbol table, dan lexical level.

```
printDecoratedAST(root):
```

```

    untuk setiap node AST
        cetak nama node

        jika ada semanticType
            cetak tipe semantik

        jika ada tabIndex
            cetak indeks TAB

        cetak lexical level

        cetak child secara rekursif

```

3.17.2 Implementasi Output Symbol Table

Symbol table dicetak dalam bentuk TAB, ATAB, dan BTAB. TAB berisi informasi identifier, ATAB berisi informasi array, sedangkan BTAB berisi informasi block, procedure, function, dan record.

```

printStatsTables():
    cetak "=== TAB ==="
    cetak identifier, obj, type, lev, ref, adr

    cetak "=== ATAB ==="
    cetak indexType, elementType, low, high, elementSize, totalSize

    cetak "=== BTAB ==="
    cetak last, lpar, psze, vsze

```

3.17.3 Implementasi Output Error

Semantic error disimpan dalam daftar error agar program dapat menampilkan lebih dari satu error dalam satu kali proses analisis. Error yang sama juga dihindari agar output tidak terlalu berulang.

```

reportError(pesan):
    jika pesan belum pernah dicatat
        masukkan pesan ke daftar error

printStatsErrors():
    cetak "=== Semantic Errors ==="

    untuk setiap error
        cetak error

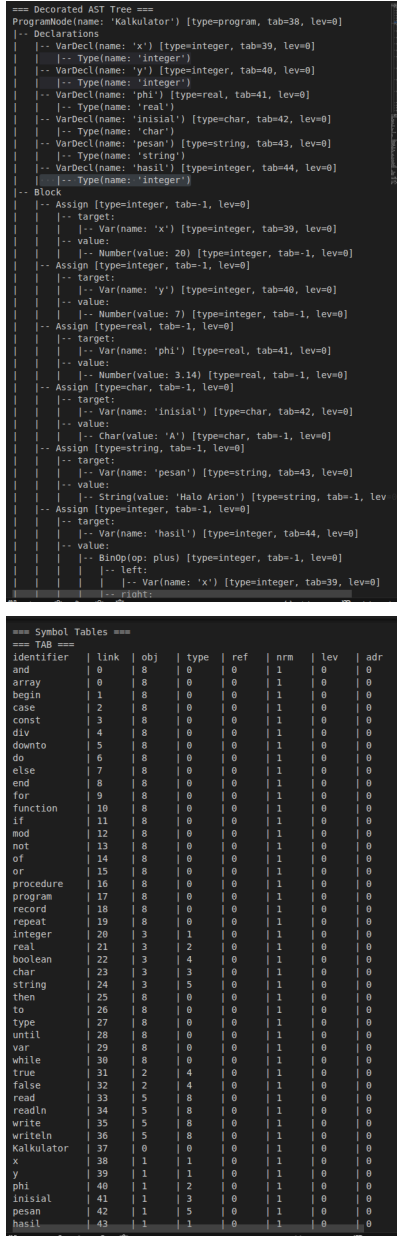
```

BAB 4 Pengujian

4.1 Pengujian Masukkan dan Keluaran Program

Karena output Decorated AST dan Symbol Table yang dihasilkan cukup panjang, tangkapan layar penuh tidak ditampilkan secara keseluruhan pada laporan agar tetap terbaca. Oleh karena itu, laporan

hanya menampilkan ringkasan input, nama file output, dan status hasil pengujian. File input, file output lengkap, serta screenshot hasil eksekusi program dapat dilihat pada repository GitHub yang tercantum pada lampiran.

Masukkan	Keluaran
Test 1 - Nama File : input-1.txt {pengujian 1 - bagian tipe data primitif dan operator aritmatika} <pre> program Kalkulator; var x, y : integer; phi : real; inisial : char; pesan : string; hasil : integer; begin (* isi nilai awal untuk setiap variabel *) x := 20; y := 7; phi := 3.14; inisial := 'A'; pesan := 'Halo Arion'; { hitung hasil penjumlahan, pengurangan, dan perkalian } hasil := x + y; hasil := x - y; hasil := x * y; (* pembagian real: 22 dibagi 7 mendekati nilai phi *) phi := 22 / 7; { tampilkan semua hasil ke layar } writeln(hasil); writeln(phi); writeln(inisial); writeln(pesan); end. </pre>	Keluaran File :output-1.txt  The screenshot shows two parts of the compiler's output. The top part is the 'Decorated AST Tree' for the program 'Kalkulator'. It details the declarations of variables x, y, phi, inisial, pesan, and hasil, followed by a block of assignments and calculations. The bottom part is the 'Symbol Tables' section, which includes a table listing identifiers, their links, object types, references, names, levels, and addresses. The table lists various keywords and identifiers used in the program, such as 'and', 'array', 'begin', 'case', 'const', 'div', 'downto', 'do', 'else', 'end', 'for', 'function', 'if', 'mod', 'not', 'of', 'or', 'procedure', 'program', 'record', 'repeat', 'integer', 'real', 'boolean', 'char', 'string', 'then', 'to', 'type', 'until', 'var', 'while', 'true', 'false', 'read', 'readln', 'write', 'writeln', 'Kalkulator', 'x', 'y', 'phi', 'inisial', 'pesan', and 'hasil'.
Test 2 - Nama File :input-2.txt (* pengujian 2 - bagian percabangan dan	Keluaran File :output-2.txt

Seterusnya dapat dilihat di repository pada lampiran

```
operator logika *)

program CekNilai;
var
    nilai : integer;
    lulus : integer;
begin
    nilai := 75;
    lulus := 0;

    { bandingkan nilai dengan beberapa ambang
batas menggunakan operator relasional }
    if nilai > 90 then
        writeln('Sangat Baik')
    else if nilai >= 75 then
        writeln('Baik')
    else if nilai < 50 then
        writeln('Kurang')
    else if nilai <= 59 then
        writeln('Cukup')
    else
        writeln('Sedang');

    (* pastikan nilai tidak sama dengan nol
sebelum diproses lebih lanjut *)
    if nilai <> 0 then
        writeln('Nilai tidak nol');

    { operator NOT membalik nilai logika dari
kondisi yang dibungkus kurung }
    if NOT (nilai < 0) then
        lulus := 1;

    (* operator AND mensyaratkan kedua sisi
bernilai benar secara bersamaan *)
    if (nilai >= 75) AND (nilai <= 100) then
        writeln('Lulus');

    { cek apakah nilai di luar rentang yang
diperbolehkan }
    if (nilai < 0) OR (nilai > 100) then
        writeln('Nilai tidak valid');

    (* operator == diuji di sini, perhatikan
jika ada bug pada EQL_STATE di lexer *)
    if nilai == 75 then
        writeln('Tepat 75');

end.
```

```
-- Decoded AST Tree ==
ProgramNode(name: 'Gekhilai') [type=program, tab=38, lev=0]
-- Declarations
|-- VarDecl(name: 'nilai') [type=integer, tab=39, lev=0]
|   |-- Type(name: 'integer')
|   -- VarDecl(name: 'lulus') [type=integer, tab=40, lev=0]
|       |-- Type(name: 'integer')
-- Block
|-- Assign [type=integer, tab=1, lev=0]
|   |-- target:
|   |   |-- Var(name: 'nilai') [type=integer, tab=39, lev=0]
|   |   -- value:
|   |       |-- Number(value: 75) [type=integer, tab=-1, lev=0]
|   -- Assign [type=integer, tab=-1, lev=0]
|       |-- target:
|       |   |-- Var(name: 'lulus') [type=integer, tab=40, lev=0]
|       |   -- value:
|       |       |-- Number(value: 0) [type=integer, tab=-1, lev=0]
|       -- If [type=boolean, tab=1, lev=0]
|           |-- condition:
|           |   |-- BinOp(op: gtr) [type=boolean, tab=-1, lev=0]
|           |   |   |-- left:
|           |   |   |   |-- Var(name: 'nilai') [type=integer, tab=39, lev=0]
|           |   |   |   -- right:
|           |   |       |-- Number(value: 90) [type=integer, tab=-1, lev=0]
|           |   -- then:
|           |       |-- ProcCall(name: 'writeln') [type=procedure, tab=37, lev=0]
|           |       |   |-- String(value: 'Sangat Baik!') [type=string, tab=-1]
|           |       -- else:
|           |           |-- If [type=boolean, tab=1, lev=0]
|           |               |-- condition:
|           |               |   |-- BinOp(op: geq) [type=boolean, tab=-1, lev=0]
|           |               |   |   |-- left:
|           |               |   |   |   |-- Var(name: 'nilai') [type=integer, tab=-1]
|           |               |   |   |   -- right:
|           |               |   |       |-- Number(value: 75) [type=integer, tab=-1]
|           |               -- then:
|           |                   |-- ProcCall(name: 'writeln') [type=procedure, tab=-1]
|           |                   |   |-- String(value: 'Baik') [type=string, tab=-1]
|           |                   -- else:
|           |                       |-- If [type=boolean, tab=1, lev=0]
|           |                           |-- condition:
|           |                           |   |-- BinOp(op: lss) [type=boolean, tab=-1, lev=0]
|           |                           |   |   |-- left:
|           |                           |   |   |   |-- Var(name: 'nilai') [type=integer, tab=-1]
|           |                           |   |   |   -- right:
|           |                           |   |       |-- Number(value: 50) [type=integer, tab=-1]
|           |                           -- then:
|           |                               |-- ProcCall(name: 'writeln') [type=procedure, tab=-1]
```

=== Symbol Tables ===							
=== TAB ===							
identifier	link	obj	type	ref	nrm	lev	adr
and	0	8	0	0	1	0	0
array	0	8	0	0	1	0	0
begin	1	8	0	0	1	0	0
case	2	8	0	0	1	0	0
const	3	8	0	0	1	0	0
div	4	8	0	0	1	0	0
downto	5	8	0	0	1	0	0
do	6	8	0	0	1	0	0
else	7	8	0	0	1	0	0
end	8	8	0	0	1	0	0
for	9	8	0	0	1	0	0
function	10	8	0	0	1	0	0
if	11	8	0	0	1	0	0
mod	12	8	0	0	1	0	0
not	13	8	0	0	1	0	0
of	14	8	0	0	1	0	0
or	15	8	0	0	1	0	0
procedure	16	8	0	0	1	0	0
program	17	8	0	0	1	0	0
record	18	8	0	0	1	0	0
repeat	19	8	0	0	1	0	0
integer	20	3	1	0	1	0	0
real	21	3	2	0	1	0	0
boolean	22	3	4	0	1	0	0
char	23	3	3	0	1	0	0
string	24	3	5	0	1	0	0
then	25	8	0	0	1	0	0
to	26	8	0	0	1	0	0
type	27	8	0	0	1	0	0
until	28	8	0	0	1	0	0
var	29	8	0	0	1	0	0
while	30	8	0	0	1	0	0
true	31	2	4	0	1	0	0
false	32	2	4	0	1	0	0
read	33	5	8	0	1	0	0
readln	34	5	8	0	1	0	0
write	35	5	8	0	1	0	0
writeln	36	5	8	0	1	0	0
CekNilai	37	0	0	0	1	0	0
nilai	38	1	1	0	1	0	0
lulus	39	1	1	0	1	0	0
=== ATAB ===							
indexType	elementType	elementRef	low	high	elemen		

Seterusnya dapat dilihat di repository pada lampiran

Test 3 - Nama File :input-3.txt

Keluaran File :output-3.txt

```
{pengujian 3 - bagian perulangan dengan for,
while, dan repeat}
```

```
program Perulangan;
```

```
var
```

```
    i, n, total : integer;
```

```
begin
```

```
    n      := 5;
```

```
    total := 0;
```

```
    { jumlahkan angka 1 sampai n menggunakan
    perulangan for naik }
```

```
    for i := 1 to n do
```

```
    begin
```

```
        total := total + i;
```

```
    end;
```

```
    writeln(total);
```

```
    (* cetak angka dari n turun ke 1
    menggunakan for downto *)
```

```
    for i := n downto 1 do
```

```
    begin
```

```
        writeln(i);
```

```
    end;
```

```
    { perulangan while akan terus berjalan
    selama kondisinya benar }
```

```
    i := 1;
```

```
    while i <= n do
```

```
    begin
```

```
        writeln(i);
```

```
        i := i + 1;
```

```
    end;
```

```
    (* repeat dijalankan minimal satu kali
    sebelum kondisi until diperiksa )
```

```
    i := 1;
```

```
    repeat
```

```
        writeln(i);
```

```
        i := i + 1;
```

```
    until i > n;
```

```
end.
```

```
=== Decorated AST Tree ===
ProgramNode(name: 'Perulangan') [type=program, tab=38, lev=0]
-- Declarations
-- VarDecl(name: 'i') [type=integer, tab=39, lev=0]
-- Type(name: 'integer')
-- VarDecl(name: 'n') [type=integer, tab=40, lev=0]
-- Type(name: 'integer')
-- VarDecl(name: 'total') [type=integer, tab=41, lev=0]
-- Type(name: 'integer')
-- Block
-- Assign [type=integer, tab=-1, lev=0]
-- target:
-- Var(name: 'n') [type=integer, tab=40, lev=0]
-- value:
-- Number(value: 5) [type=integer, tab=-1, lev=0]
-- Assign [type=integer, tab=-1, lev=0]
-- target:
-- Var(name: 'total') [type=integer, tab=41, lev=0]
-- value:
-- Number(value: 0) [type=integer, tab=-1, lev=0]
-- For [type=integer, tab=-1, lev=0]
-- Assign [type=integer, tab=-1, lev=0]
-- target:
-- Var(name: 'i') [type=integer, tab=39, lev=0]
-- value:
-- Number(value: 1) [type=integer, tab=-1, lev=0]
-- to:
-- Var(name: 'n') [type=integer, tab=40, lev=0]
-- body:
-- Block
-- Assign [type=integer, tab=-1, lev=0]
-- target:
-- Var(name: 'total') [type=integer, tab=41, lev=0]
-- value:
-- BinOp(op: plus) [type=integer, tab=-1, lev=0]
-- left:
-- Var(name: 'total') [type=integer, tab=41, lev=0]
-- right:
-- Var(name: 'i') [type=integer, tab=39, lev=0]
-- ProcCall(name: 'writeln') [type=procedure, tab=37, lev=0]
-- Var(name: 'total') [type=integer, tab=41, lev=0]
-- For [type=integer, tab=-1, lev=0]
-- Assign [type=integer, tab=-1, lev=0]
-- target:
-- Var(name: 'i') [type=integer, tab=39, lev=0]
-- value:
-- Var(name: 'n') [type=integer, tab=40, lev=0]
-- to:
-- Number(value: 1) [type=integer, tab=-1, lev=0]
```

```
=== Symbol Tables ===
=== TAB ===
identifier | link | obj | type | ref | nrm | lev | adr
and         | 0   | 8   | 0   | 0   | 1   | 0   | 0
array       | 0   | 8   | 0   | 0   | 1   | 0   | 0
begin       | 1   | 8   | 0   | 0   | 1   | 0   | 0
case        | 2   | 8   | 0   | 0   | 1   | 0   | 0
const       | 3   | 8   | 0   | 0   | 1   | 0   | 0
div         | 4   | 8   | 0   | 0   | 1   | 0   | 0
downto      | 5   | 8   | 0   | 0   | 1   | 0   | 0
do          | 6   | 8   | 0   | 0   | 1   | 0   | 0
else        | 7   | 8   | 0   | 0   | 1   | 0   | 0
end         | 8   | 8   | 0   | 0   | 1   | 0   | 0
for         | 9   | 8   | 0   | 0   | 1   | 0   | 0
function    | 10  | 8   | 0   | 0   | 1   | 0   | 0
if          | 11  | 8   | 0   | 0   | 1   | 0   | 0
mod         | 12  | 8   | 0   | 0   | 1   | 0   | 0
not         | 13  | 8   | 0   | 0   | 1   | 0   | 0
of          | 14  | 8   | 0   | 0   | 1   | 0   | 0
or          | 15  | 8   | 0   | 0   | 1   | 0   | 0
procedure   | 16  | 8   | 0   | 0   | 1   | 0   | 0
program     | 17  | 8   | 0   | 0   | 1   | 0   | 0
record      | 18  | 8   | 0   | 0   | 1   | 0   | 0
repeat      | 19  | 8   | 0   | 0   | 1   | 0   | 0
integer     | 20  | 3   | 1   | 0   | 1   | 0   | 0
real        | 21  | 3   | 2   | 0   | 1   | 0   | 0
boolean     | 22  | 3   | 4   | 0   | 1   | 0   | 0
char        | 23  | 3   | 3   | 0   | 1   | 0   | 0
string      | 24  | 3   | 5   | 0   | 1   | 0   | 0
then        | 25  | 8   | 0   | 0   | 1   | 0   | 0
to          | 26  | 8   | 0   | 0   | 1   | 0   | 0
type        | 27  | 8   | 0   | 0   | 1   | 0   | 0
until       | 28  | 8   | 0   | 0   | 1   | 0   | 0
var         | 29  | 8   | 0   | 0   | 1   | 0   | 0
while       | 30  | 8   | 0   | 0   | 1   | 0   | 0
true        | 31  | 2   | 4   | 0   | 1   | 0   | 0
false       | 32  | 2   | 4   | 0   | 1   | 0   | 0
read        | 33  | 5   | 8   | 0   | 1   | 0   | 0
readln      | 34  | 5   | 8   | 0   | 1   | 0   | 0
write       | 35  | 5   | 8   | 0   | 1   | 0   | 0
writeln     | 36  | 5   | 8   | 0   | 1   | 0   | 0
Perulangan  | 37  | 0   | 0   | 0   | 1   | 0   | 0
i           | 38  | 1   | 1   | 0   | 1   | 0   | 0
n           | 39  | 1   | 1   | 0   | 1   | 0   | 0
total       | 40  | 1   | 1   | 0   | 1   | 0   | 0

=== ATAB ===
indexType | elementType | elementRef | low | high | elemen
```

Seterusnya dapat dilihat di repository pada lampiran

Test 4 - Nama File :input-4.txt

(* pengujian 4 - bagian deklarasi tipe kompleks, fungsi, dan prosedur *)

```

program TipeKompleks;
const
    MAX == 10;
    PI  == 3.14159;
type
    Titik == record
        x : real;
        y : real;
    end;
    Daftar == array [1..10] of integer;
var
    p      : Titik;
    data   : Daftar;
    i      : integer;

{ fungsi ini menerima satu bilangan bulat dan
mengembalikan kuadratnya }
function Kuadrat(n : integer) : integer;
begin
    Kuadrat := n * n;
end;

(* prosedur ini mengisi array dengan nilai
kuadrat dari indeks nya *)
procedure IsiArray(arr : Daftar; n : integer);
var
    j : integer;
begin
    for j := 1 to n do
        begin
            arr[j] := j * j;
        end;
    end;

begin
    { isi koordinat titik dengan nilai real }
    p.x := 3.0;
    p.y := 4.0;

    (* panggil prosedur untuk mengisi data,
    lalu tampilkan hasilnya *)
    IsiArray(data, MAX);

    for i := 1 to MAX do
        begin
            writeln(data[i]);
        end;
    end;

```

Keluaran File :output-4.txt

```

=== Decorated AST Tree ===
ProgramNode(name: 'TipeKompleks') [type=program, tab=38, lev=0]
-- Declarations
  -- ConstDecl(name: 'MAX') [type=integer, tab=39, lev=0]
    |-- Number(value: 10) [type=integer, tab=-1, lev=0]
  -- ConstDecl(name: 'PI') [type=real, tab=40, lev=0]
    |-- Number(value: 3.14159) [type=real, tab=-1, lev=0]
  -- TypeDecl(name: 'Titik') [type=record, tab=43, lev=0]
    |-- RecordType
      |-- Field(name: 'x')
        |-- Type(name: 'real')
      |-- Field(name: 'y')
        |-- Type(name: 'real')
    |-- TypeDecl(name: 'Daftar') [type=array, tab=44, lev=0]
      |-- ArrayType
        |-- index:
          |-- RangeType
            |-- first:
              |-- Number(value: 1)
            |-- last:
              |-- Number(value: 10)
            |-- base:
              |-- Type(name: 'integer')
          |-- element:
            |-- Type(name: 'integer')
  -- VarDecl(name: 'p') [type=titik, tab=45, lev=0]
    |-- Type(name: 'Titik')
  -- VarDecl(name: 'data') [type=daftar, tab=46, lev=0]
    |-- Type(name: 'Daftar')
  -- VarDecl(name: 'i') [type=integer, tab=47, lev=0]
    |-- Type(name: 'integer')
  -- FunctionDecl(name: 'Kuadrat') [type=integer, tab=48, lev=0]
    |-- returnType:
      |-- Type(name: 'integer')
    |-- Parameter(name: 'n') [type=integer, tab=50, lev=1]
      |-- Type(name: 'integer')
    |-- Declarations
      |-- Block
        |-- Assign [type=integer, tab=-1, lev=1]
          |-- target:
            |-- Var(name: 'Kuadrat') [type=integer, tab=49, lev=1]
          |-- value:
            |-- BinOp(op: times) [type=integer, tab=-1, lev=1]
              |-- left:
                |-- Var(name: 'n') [type=integer, tab=50, lev=1]
              |-- right:
                |-- Var(name: 'n') [type=integer, tab=50, lev=1]
  -- ProcedureDecl(name: 'IsiArray') [type=procedure, tab=51, lev=0]
    |-- Parameter(name: 'arr') [type=daftar, tab=52, lev=1]

```

```

=== Symbol Tables ===
=== TAB ===

```

identifier	link	obj	type	ref	nrm	lev	ad
and	0	8	0	0	1	0	0
array	0	8	0	0	1	0	0
begin	1	8	0	0	1	0	0
case	2	8	0	0	1	0	0
const	3	8	0	0	1	0	0
div	4	8	0	0	1	0	0
downto	5	8	0	0	1	0	0
do	6	8	0	0	1	0	0
else	7	8	0	0	1	0	0
end	8	8	0	0	1	0	0
for	9	8	0	0	1	0	0
function	10	8	0	0	1	0	0
if	11	8	0	0	1	0	0
mod	12	8	0	0	1	0	0
not	13	8	0	0	1	0	0
of	14	8	0	0	1	0	0
or	15	8	0	0	1	0	0
procedure	16	8	0	0	1	0	0
program	17	8	0	0	1	0	0
record	18	8	0	0	1	0	0
repeat	19	8	0	0	1	0	0
integer	20	3	1	0	1	0	0
real	21	3	2	0	1	0	0
boolean	22	3	4	0	1	0	0
char	23	3	3	0	1	0	0
string	24	3	5	0	1	0	0
then	25	8	0	0	1	0	0
to	26	8	0	0	1	0	0
type	27	8	0	0	1	0	0
until	28	8	0	0	1	0	0
var	29	8	0	0	1	0	0
while	30	8	0	0	1	0	0
true	31	2	4	0	1	0	0
false	32	2	4	0	1	0	0
read	33	5	8	0	1	0	0
readln	34	5	8	0	1	0	0
write	35	5	8	0	1	0	0
writeln	36	5	8	0	1	0	0
TipeKompleks	37	0	0	0	1	0	0
MAX	38	2	1	0	1	0	0
PI	39	2	2	0	1	0	0
x	40	7	2	0	1	0	0
y	41	7	2	0	1	0	1
Titik	42	3	7	1	1	0	0

Seterusnya dapat dilihat di repository pada lampiran


```
writeln(Kuadrat(5));  
end.
```

Test 5 - Nama File :input-5.txt

```
{pengujian 5 - bagian case, operator div dan  
mod, serta variasi komentar}
```

```

program KasusLengkap;
var
    angka    : integer;
    bagi     : integer;
    sisa     : integer;
    harakat  : char;
begin
    angka := 17;

    { bagi integer: 17 dibagi 5 hasilnya 3,
    bukan 3.4 }
    bagi := angka div 5;

    (* sisa bagi: 17 mod 5 hasilnya 2 karena 5
    masuk 3 kali dengan sisa 2 *)
    sisa := angka MOD 5;

    writeln(bagi);
    writeln(sisa);

    { gunakan case of untuk memeriksa nilai
    sisa pembagian satu per satu }
    case sisa of
        0 : writeln('Habis dibagi 5');
        1 : writeln('Sisa 1');
        2 : writeln('Sisa 2');
        3 : writeln('Sisa 3');
        4 : writeln('Sisa 4');
    end;

    (* simpan satu karakter huruf ke dalam
    variabel bertipe char *)
    harakat := 'Z';

    writeln(harakat);
end.

```

Keluaran File :output-5.txt

```

== Decorated AST Tree ==
ProgramNode(name: 'KasusLengkap') [type=program, tab=38, lev=0]
-- Declarations
  -- VarDecl(name: 'angka') [type=integer, tab=39, lev=0]
    -- Type(name: 'integer')
  -- VarDecl(name: 'bagi') [type=integer, tab=40, lev=0]
    -- Type(name: 'integer')
  -- VarDecl(name: 'sisa') [type=integer, tab=41, lev=0]
    -- Type(name: 'integer')
  -- VarDecl(name: 'harakat') [type=char, tab=42, lev=0]
    -- Type(name: 'char')
-- Block
  -- Assign [type=integer, tab=-1, lev=0]
    -- target:
      -- Var(name: 'angka') [type=integer, tab=39, lev=0]
    -- value:
      -- Number(value: 17) [type=integer, tab=-1, lev=0]
  -- Assign [type=integer, tab=-1, lev=0]
    -- target:
      -- Var(name: 'bagi') [type=integer, tab=40, lev=0]
    -- value:
      -- BinOp(op: idiv) [type=integer, tab=-1, lev=0]
        -- left:
          -- Var(name: 'angka') [type=integer, tab=39, lev=0]
        -- right:
          -- Number(value: 5) [type=integer, tab=-1, lev=0]
  -- Assign [type=integer, tab=-1, lev=0]
    -- target:
      -- Var(name: 'sisa') [type=integer, tab=41, lev=0]
    -- value:
      -- BinOp(op: imod) [type=integer, tab=-1, lev=0]
        -- left:
          -- Var(name: 'angka') [type=integer, tab=39, lev=0]
        -- right:
          -- Number(value: 5) [type=integer, tab=-1, lev=0]
  -- ProcCall(name: 'writeln') [type=procedure, tab=37, lev=0]
    -- Var(name: 'bagi') [type=integer, tab=40, lev=0]
  -- ProcCall(name: 'writeln') [type=procedure, tab=37, lev=0]
    -- Var(name: 'sisa') [type=integer, tab=41, lev=0]
  -- Case [type=integer, tab=-1, lev=0]
    -- selector:
      -- Var(name: 'sisa') [type=integer, tab=41, lev=0]
    -- CaseBlock [type=integer, tab=-1, lev=0]
      -- Number(value: 0) [type=integer, tab=-1, lev=0]
        -- ProcCall(name: 'writeln') [type=procedure, tab=37, lev=0]
          -- String(value: 'Habis dibagi 5') [type=string, tab=-1, lev=0]
      -- CaseBlock [type=integer, tab=-1, lev=0]
        -- Number(value: 1) [type=integer, tab=-1, lev=0]
          -- ProcCall(name: 'writeln') [type=procedure, tab=37, lev=0]
            -- Char(value: 'Z') [type=char, tab=-1, lev=0]
          -- ProcCall(name: 'writeln') [type=procedure, tab=37, lev=0]
            -- Var(name: 'harakat') [type=char, tab=42, lev=0]

```

```

== Symbol Tables ==
== TAB ==
identifier | link | obj | type | ref | nrm | lev | ad
and         | 0   | 8   | 0   | 0   | 1   | 0   | 0
array       | 0   | 8   | 0   | 0   | 1   | 0   | 0
begin       | 1   | 8   | 0   | 0   | 1   | 0   | 0
case        | 2   | 8   | 0   | 0   | 1   | 0   | 0
const       | 3   | 8   | 0   | 0   | 1   | 0   | 0
div         | 4   | 8   | 0   | 0   | 1   | 0   | 0
downto      | 5   | 8   | 0   | 0   | 1   | 0   | 0
do          | 6   | 8   | 0   | 0   | 1   | 0   | 0
else        | 7   | 8   | 0   | 0   | 1   | 0   | 0
end         | 8   | 8   | 0   | 0   | 1   | 0   | 0
for         | 9   | 8   | 0   | 0   | 1   | 0   | 0
function    | 10  | 8   | 0   | 0   | 1   | 0   | 0
if          | 11  | 8   | 0   | 0   | 1   | 0   | 0
mod         | 12  | 8   | 0   | 0   | 1   | 0   | 0
not         | 13  | 8   | 0   | 0   | 1   | 0   | 0
of          | 14  | 8   | 0   | 0   | 1   | 0   | 0
or          | 15  | 8   | 0   | 0   | 1   | 0   | 0
procedure   | 16  | 8   | 0   | 0   | 1   | 0   | 0
program     | 17  | 8   | 0   | 0   | 1   | 0   | 0
record      | 18  | 8   | 0   | 0   | 1   | 0   | 0
repeat      | 19  | 8   | 0   | 0   | 1   | 0   | 0
integer     | 20  | 3   | 1   | 0   | 1   | 0   | 0
real        | 21  | 3   | 2   | 0   | 1   | 0   | 0
boolean     | 22  | 3   | 4   | 0   | 1   | 0   | 0
char        | 23  | 3   | 3   | 0   | 1   | 0   | 0
string      | 24  | 3   | 5   | 0   | 1   | 0   | 0
then        | 25  | 8   | 0   | 0   | 1   | 0   | 0
to          | 26  | 8   | 0   | 0   | 1   | 0   | 0
type        | 27  | 8   | 0   | 0   | 1   | 0   | 0
until       | 28  | 8   | 0   | 0   | 1   | 0   | 0
var         | 29  | 8   | 0   | 0   | 1   | 0   | 0
while       | 30  | 8   | 0   | 0   | 1   | 0   | 0
true        | 31  | 2   | 4   | 0   | 1   | 0   | 0
false       | 32  | 2   | 4   | 0   | 1   | 0   | 0
read        | 33  | 5   | 8   | 0   | 1   | 0   | 0
readln      | 34  | 5   | 8   | 0   | 1   | 0   | 0
write       | 35  | 5   | 8   | 0   | 1   | 0   | 0
writeln     | 36  | 5   | 8   | 0   | 1   | 0   | 0
KasusLengkap | 37 | 0 | 0 | 0 | 1 | 0 | 0
angka      | 38  | 1   | 1   | 0   | 1   | 0   | 0
bagi       | 39  | 1   | 1   | 0   | 1   | 0   | 0

```

Seterusnya dapat dilihat di repository pada lampiran

Test 6 - Nama File :input-6.txt

```
{pengujian 6 - identifier digunakan sebelum
dideklarasikan}
```

```
program IdentifierBelumDeklarasi;
begin
    nilai := 10;
    writeln(nilai);
end.
```

Keluaran File :output-6.txt

```

=== Semantic Errors ===
Semantic error: identifier 'nilai' is not declared.

=== Decorated AST Tree ===
ProgramNode(name: 'IdentifierBelumDeklarasi') [type=program, tab=38,
|-- Declarations
|-- Block
|   |-- Assign
|   |   |-- target:
|   |   |   |-- Var(name: 'nilai')
|   |   |-- value:
|   |   |   |-- Number(value: 10) [type=integer, tab=-1, lev=0]
|   |-- ProcCall(name: 'writeln') [type=procedure, tab=37, lev=0]
|   |   |-- Var(name: 'nilai')

=== Symbol Tables ===
=== TAB ===

```

identifier	link	obj	type	ref	nrm
and	0	8	0	0	1
array	0	8	0	0	1
begin	1	8	0	0	1
case	2	8	0	0	1
const	3	8	0	0	1
div	4	8	0	0	1
downto	5	8	0	0	1
do	6	8	0	0	1
else	7	8	0	0	1
end	8	8	0	0	1
for	9	8	0	0	1
function	10	8	0	0	1
if	11	8	0	0	1
mod	12	8	0	0	1
not	13	8	0	0	1
of	14	8	0	0	1
or	15	8	0	0	1
procedure	16	8	0	0	1
program	17	8	0	0	1
record	18	8	0	0	1
repeat	19	8	0	0	1
integer	20	3	1	0	1
real	21	3	2	0	1
boolean	22	3	4	0	1
char	23	3	3	0	1
string	24	3	5	0	1
then	25	8	0	0	1
to	26	8	0	0	1
type	27	8	0	0	1
until	28	8	0	0	1
var	29	8	0	0	1

Seterusnya dapat dilihat di repository pada lampiran

Test 7 - Nama File :input-7.txt

{pengujian 7 - type mismatch pada assignment dan kondisi non-boolean}

```
program KesalahanTipe;
var
    angka    : integer;
    teks     : string;
    status   : boolean;
begin
    angka := 'Halo';
    teks := 123;
    status := angka;

    if angka then
        begin
            writeln(angka);
        end;
    end.
end.
```

Keluaran File :output-7.txt

Seterusnya dapat dilihat di repository pada lampiran

Seterusnya dapat dilihat di repository pada lampiran

BAB 5

Kesimpulan dan Saran

5.1 Kesimpulan

Tahap semantic analysis merupakan langkah lanjutan yang penting setelah proses lexical analysis dan syntax analysis dalam pembuatan compiler bahasa pemrograman Arion. Pada tahap ini, program tidak hanya memeriksa apakah source code sudah sesuai dengan aturan grammar, tetapi juga memeriksa apakah program memiliki makna yang benar. Dengan adanya semantic analysis, program dapat mendeteksi kesalahan seperti penggunaan identifier yang belum dideklarasikan, deklarasi ulang pada scope yang sama, assignment dengan tipe yang tidak kompatibel, penggunaan kondisi non-boolean pada struktur kontrol, serta kesalahan pada penggunaan array, record, subrange, enumerated type, prosedur, dan fungsi. Parse tree yang dihasilkan oleh parser berhasil digunakan sebagai dasar untuk membentuk Abstract Syntax Tree atau AST. AST dibuat lebih ringkas dibandingkan parse tree karena hanya menyimpan informasi yang penting untuk proses analisis semantik. Informasi sintaksis yang tidak diperlukan, seperti beberapa keyword, tanda baca, dan node grammar perantara, tidak lagi dijadikan bagian utama dalam AST. Dengan bentuk AST yang lebih sederhana, proses pengecekan semantik dapat dilakukan dengan lebih terarah.

Selain pembentukan AST, program juga berhasil menghasilkan Decorated AST. Decorated AST merupakan AST yang telah diberi anotasi tambahan hasil dari semantic analysis. Anotasi tersebut berupa tipe data, referensi ke symbol table, dan lexical level. Informasi ini membantu menunjukkan hubungan antara penggunaan identifier pada program dengan deklarasinya, serta memperlihatkan tipe data dari setiap node penting pada AST. Dengan demikian, Decorated AST dapat digunakan sebagai bukti bahwa proses analisis semantik telah dilakukan terhadap struktur program. Program juga telah mengimplementasikan symbol table yang terdiri dari TAB, ATAB, dan BTAB. TAB digunakan untuk menyimpan informasi identifier, ATAB digunakan untuk menyimpan informasi array, sedangkan BTAB digunakan untuk menyimpan informasi block, procedure, function, dan record. Symbol table ini berperan penting dalam proses pengecekan deklarasi, scope resolution, type checking, serta validasi tipe terstruktur. Reserved words dan predefined identifiers juga dimasukkan ke dalam symbol table agar dapat dikenali sejak awal proses analisis.

5.2 Saran

Pada pengembangan berikutnya, semantic analyzer masih dapat ditingkatkan agar mampu menangani aturan semantik bahasa Arion secara lebih lengkap. Salah satunya adalah memperjelas mekanisme pelaporan error dengan menambahkan informasi posisi baris dan kolom pada source code. Dengan adanya informasi tersebut, pengguna dapat lebih mudah mengetahui lokasi kesalahan semantik yang terjadi dan memperbaikinya secara langsung.

Lampiran

Link Github : <https://github.com/Alpaomega1136/JML-Tubes-IF2224-2026.git>

Pembagian Tugas :

NIM	Nama	Pengerjaan pada milestone ini	Persentase
13524015	Mahatma Brahmana	Semantic analyzer, type checking, scope, dan validasi symbol, laporan bab 3	25%
13524057	Benedict Darrel Setiawan	Foundation struktur data AST dan symbol table, laporan bab 3	25%
13524059	Raymond Jonathan Dwi Putra J	Laporan, output integration, AST printer, test case, dokumentasi, dan beberapa revisi	25%
13524111	Reynard Anderson Wijaya	AST builder dari parse tree ke AST, laporan bab 3	25%

Daftar Pustaka

- Cornell University. (2022). *Semantic analysis*. CS 4120: Introduction to Compilers. <https://www.cs.cornell.edu/courses/cs4120/2022sp/notes/semantic/>
- GeeksforGeeks. (2025). *Semantic analysis in compiler design*. <https://www.geeksforgeeks.org/compiler-design/semantic-analysis-in-compiler-design/>
- GeeksforGeeks. (2025). *Symbol table in compiler*. <https://www.geeksforgeeks.org/symbol-table-compiler/>
- Labouseur, A. G. (n.d.). *Semantic analysis* [PDF]. <https://www.labouseur.com/courses/compilers/semantic-analysis.pdf>
- Nystrom, R. (2021). *Crafting interpreters*. <https://craftinginterpreters.com/>
- Nystrom, R. (2021). *Crafting interpreters* [Source code repository]. GitHub. <https://github.com/munificent/craftinginterpreters>
- Python Software Foundation. (2026). *ast: Abstract syntax trees*. Python 3 Documentation. <https://docs.python.org/3/library/ast.html>
- Spivak, R. (n.d.). *Let's build a simple interpreter*. <https://ruslanspivak.com/lsbasi-part1/>
- Compilers: Principles, Techniques, and Tools. (2006). *Compilers: Principles, techniques, and tools*. Pearson.